

SC-JEPA: Stabilizing Latent Predictive Learning for Time-Series Anomaly Prediction

Yanan He¹ Yunshi Wen² Xin Wang³
Tengfei Ma³

Abstract

Time-series anomaly prediction aims to forecast future system failures before they fully emerge, making latent predictive models such as JEPA a promising framework for capturing precursor dynamics. However, directly applying continuous self-distillation to time-series data is often unstable and can lead to representation collapse, while also struggling to model precursors evolving at different temporal scales. To address this, we propose **SC-JEPA**, a new JEPA-based framework to model time-series anomaly prediction in a discretized predictive state space. It introduces a soft codebook bottleneck to stabilize latent predictive learning and encourage regime-level structure in the learned representations. Building on this stabilized latent space, we further design a multi-resolution predictive objective to capture precursor patterns at different temporal scales. Experiments on five real-world benchmarks show that SC-JEPA achieves strong and consistent early-warning performance. The source code is publicly available at <https://github.com/Echoo113/SC-JEPA>.

1. Introduction

Multivariate time series underpin many safety-critical systems, from industrial plants to server operations, where failures can propagate rapidly and incur substantial costs. In such settings, the key challenge is not merely detecting anomalies after they occur, but anticipating them early enough for intervention. This task, known as *anomaly prediction*, differs fundamentally from both anomaly detection and forecasting (Park et al., 2025). Detection is inherently reactive and identifies faults only after they manifest (Ruff et al., 2018; Xu et al., 2022), while forecasting focuses on

¹ Yale University ² Rensselaer Polytechnic Institute ³ Stony Brook University . Correspondence to: Tengfei Ma <tengfei.ma@stonybrook.edu>.

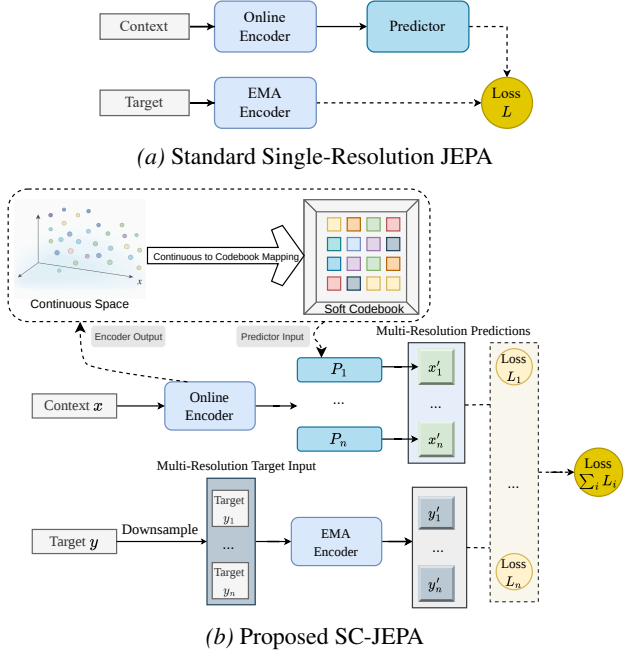


Figure 1. Comparison between the standard single-resolution JEPA and our proposed multi-resolution JEPA with a soft codebook.

regressing future observations or values (Zhou et al., 2021; Nie et al., 2023). Anomaly prediction, however, aims to isolate latent precursor signals within the current window that foreshadow impending instability (Jhin et al., 2023; Zhao et al., 2024; Park et al., 2025). Consequently, the primary objective is capturing the evolving system risk rather than accurately reproducing the exact future trajectory.

A natural approach to this problem is to model how system states evolve over time. Joint-Embedding Predictive Architectures (JEPA) (Assran et al., 2023) offer a compelling paradigm by learning to predict future representations directly in latent space. This objective aligns well with the mechanics of anomaly prediction: impending failures are often characterized by subtle regime shifts and semantic state drifts, rather than deterministic variations in raw signal amplitude. By optimizing latent predictability, JEPA provides an inherently task-aligned framework for anticipating system instability (Ennadir et al., 2025).

However, directly applying JEPA-style predictive learning to anomaly prediction faces two critical bottlenecks. The primary obstacle is a severe vulnerability to representation collapse (Chen & He, 2021; Grill et al., 2020; Jing et al., 2022; Bardes et al., 2022). First, learning in a continuous latent space without explicit constraints can lead to representation collapse or instability. The encoder may map diverse inputs to a constant vector or experience partial dimensional collapse, stripping the feature space of discriminative capacity (Tian et al., 2021). The continuous and noisy nature of time series data further exacerbates this instability. Moreover, in real-world systems, anomaly precursors typically emerge across heterogeneous time scales, ranging from abrupt shocks to slowly accumulating drifts (Wang et al., 2024; Liu et al., 2022a; Chen et al., 2024; Wang et al., 2023). A rigid single-scale predictive design (Figure 1a) therefore struggles to capture the full spectrum of precursor dynamics that precede anomalous events. In this work, we argue that these challenges stem from a common source: the lack of structure in the predictive latent space. When representations are unconstrained, the model must simultaneously learn stable dynamics, meaningful abstractions, and scale-aware patterns, leading to degenerate solutions.

To address these challenges, we propose **SC-JEPA** (Soft Codebook JEPA), a framework that introduces a discretized predictive state space for time-series anomaly prediction. The key idea of SC-JEPA is to constrain latent prediction through a *soft codebook bottleneck*, which maps continuous latent features to distributions over a finite set of prototypes. This mechanism serves two purposes. (i) This design imposes a structured geometry on the latent space, which empirically stabilizes training and encourages representations to organize into regime-level states. (ii) Such structure is well suited to anomaly prediction, where failures often correspond to transitions between underlying system regimes. Building on this stabilized and more structured latent space, we further introduce a *multi-resolution* predictive design (Figure 1b) to model system dynamics over different temporal scales. By jointly predicting fine-grained and coarse-grained future states, the model captures both short-term fluctuations and long-term trends, enabling more robust identification of precursor patterns.

In summary, our main contributions are as follows:

- **Latent Predictive Learning for Anomaly Prediction:** We formulate time-series anomaly prediction as a representation-space predictive modeling task, showing that latent state evolution is more informative for precursor identification than raw-value forecasting.
- **Soft Codebook as a Structural Stabilizer:** We introduce a soft codebook bottleneck that discretizes latent predictions, encouraging regime-level representations and mitigating collapse in practice.
- **Multi-Resolution Extension:** Building on the stabilized latent space induced by the soft codebook, we introduce a multi-resolution predictive objective to better capture precursor signals across temporal scales, thereby enabling earlier anomaly anticipation.

2. Related Works

Anomaly Prediction. Time-series research has largely followed two directions: forecasting and detection. Forecasting approaches primarily learn trajectory regression objectives (Zhou et al., 2021; Liu et al., 2024) and are typically evaluated on value prediction accuracy rather than early-warning separability. In contrast, detection methods score the current window and are benchmarked under reactive protocols (Ruff et al., 2018; Xu et al., 2022). Consequently, there is growing interest in moving beyond reactive detection toward early-warning settings, where the goal is to anticipate future abnormality or failure risk ahead of time (Jhin et al., 2023; Zhao et al., 2024; Park et al., 2025). However, many existing formulations remain tied to input-space objectives or detector adaptations, suggesting the value of exploring representation-space predictive models that emphasize latent dynamics over local noise, especially when precursors unfold across multiple time horizons. Another line of work related to our method is vector-quantized (VQ) representation learning (Van Den Oord et al., 2017), which has been adapted for time-series anomaly detection (Lee et al., 2023; 2024; Wen et al., 2025); however, our method differs essentially in the target task, representation granularity, and predictive framework.

Joint Embedding Predictive Architectures. Self-supervised learning for time series is often built on masked modeling, as in PatchTST (Nie et al., 2023), or contrastive learning over augmented views, as in TS2Vec (Yue et al., 2022). In parallel, Joint-Embedding Predictive Architectures (JEPA) formulate predictive world models by forecasting future representations directly in latent space, as demonstrated by I-JEPA for images and V-JEPA for videos (Assran et al., 2023; Bardes et al., 2024). By emphasizing latent predictability, JEPA encourages higher-level, state-like representations, which aligns naturally with time series as noisy observations of underlying evolving processes. Recent efforts have begun to adapt JEPA to time series, including TS-JEPA (Ennadir et al., 2025). However, JEPA-style predictive world models remain less explored for complex time-series systems, and stabilizing latent-space self-distillation objectives on continuous, non-stationary time series remains practically challenging.

Multi-Scale Dynamics. Multi-resolution modeling is widely used in representation learning to capture both global context and local detail, especially in computer vision (Lin

et al., 2017). Time series share a similar multi-scale nature, where long-term trends coexist with fast local variations. Recent time-series models such as Pyraformer (Liu et al., 2022b) and TimeMixer (Wang et al., 2024) incorporate multi-scale structure through pyramid-style attention or multi-granularity mixing. In most cases, scale is introduced to improve forecasting accuracy or efficiency, rather than to explicitly model state evolution at multiple temporal rates. A less explored direction is to incorporate multi-scale structure into predictive world models, so that latent states can be propagated across different time scales. This perspective is particularly relevant for anomaly prediction, where early signs may appear as slow drift or short-lived shocks. These observations point to multi-scale latent dynamics as a natural ingredient for early warning.

3. Method

3.1. Problem Setup

Consider a multivariate time series $\mathbf{x} \in \mathbb{R}^{T \times V}$. We partition $\mathbf{x}$ into a sequence of non-overlapping windows $\{\mathbf{W}_t\}_{t=1}^N$, where each window $\mathbf{W}_t \in \mathbb{R}^{T_w \times V}$ contains T_w time steps. Our goal is to learn a representation from the current window $\mathbf{W}_t$ to anticipate the anomaly status of the subsequent window $\mathbf{W}_{t+1}$, enabling proactive intervention by forecasting whether the next window will be anomalous.

Framework Overview. This proactive setting is difficult in practice because precursor cues are often weak, non-stationary, and may emerge at different temporal scales, making stable self-supervised prediction nontrivial. We propose a new JEPA framework to solve this challenging task. (1) To expose such scale-variant precursors, we construct dual-resolution inputs that disentangle local volatilities from global trends, described next in Section 3.2. As illustrated in Figure 2, SC-JEPA then adopts an asymmetric teacher-student design: an online context branch predicts future latent states from the current window, while a momentum-updated target branch provides supervision. The asymmetry imposes a deliberate information gap: the online branch observes only the fine-grained view, yet is supervised to match multi-scale targets produced from both fine and coarse views, encouraging scale-aware representations. (2) In addition, a soft codebook bottleneck is introduced to impose a discrete inductive bias for anomaly prediction by mapping continuous features to prototype-anchored code distributions, as formalized in Section 3.3.

3.2. Input Formulation and Multi-Scale Views

To make scale-variant precursor patterns learnable under a unified predictive objective, we build dual-scale views from each raw window. We first apply RevIN (Kim et al., 2022) to $\mathbf{W}_t$, yielding the normalized window $\hat{\mathbf{W}}_t$. The

affine statistics are cached to invert normalization for the reconstruction loss. We then construct two views at different resolutions:

Fine View (Patching). We tokenize $\hat{\mathbf{W}}_t$ into P patches of length L (where $T_w = P \cdot L$), forming the fine-grained representation:

$$\mathbf{X}_t^{\text{fine}} = \text{Patch}(\hat{\mathbf{W}}_t) \in \mathbb{R}^{P \times L \times V}. \quad (1)$$

Coarse View (Downsampling). Simultaneously, we generate a global view by averaging every P time points (Appendix A.2) to capture low-frequency trends. The resulting sequence of length L is treated as a single patch, aligning its dimensionality with the patch-based format:

$$\mathbf{X}_t^{\text{coarse}} = \text{DownAvg}_P(\hat{\mathbf{W}}_t) \in \mathbb{R}^{1 \times L \times V}. \quad (2)$$

The Online Branch processes only $\mathbf{X}_t^{\text{fine}}$, while the Target Branch encodes both resolutions at $t+1$ for supervision. This asymmetry serves as a structural inductive bias, compelling the model to encapsulate multi-scale semantics. Consequently, to bridge this deliberate information gap, we next detail the architectural machinery constructed to orchestrate the flow from partial local observations to stable global predictions.

3.3. Architecture

SC-JEPA couples a shared encoder, a soft codebook, a dual-resolution predictor, and an auxiliary decoder.

Encoder. Weak and scale-variant precursors call for representations that are predictable and comparable across resolutions. We therefore use a shared encoder $\mathcal{E}$ to embed raw windows into a compact latent space, using a channel-independent input formulation. Implemented as a residual CNN tokenizer followed by a Transformer backbone, $\mathcal{E}$ supports variable-length inputs, so both fine-grained patch sequences and a single coarse-grained token are processed with the same parameters. Thus, a single encoder maps both resolutions into the same embedding space.

- (i) **Online encoder ($\mathcal{E}_\theta$):** $\mathbf{h}_t = \mathcal{E}_\theta(\mathbf{X}_t^{\text{fine}})$. Operating under partial observation, $\mathcal{E}_\theta$ receives only the fine-grained view. It is trained to align with targets produced from both fine and coarse views (via the momentum teacher), which pressures the online representation to internalize global context from local evidence.
- (ii) **EMA encoder ($\mathcal{E}_\xi$):** A momentum-updated copy of the online encoder with holistic visibility. It processes both fine and coarse views to generate comprehensive, multi-scale targets: $\mathbf{h}_{t+1}^{\text{fine}} = \mathcal{E}_\xi(\mathbf{X}_{t+1}^{\text{fine}})$ and $\mathbf{h}_{t+1}^{\text{coarse}} = \mathcal{E}_\xi(\mathbf{X}_{t+1}^{\text{coarse}})$.

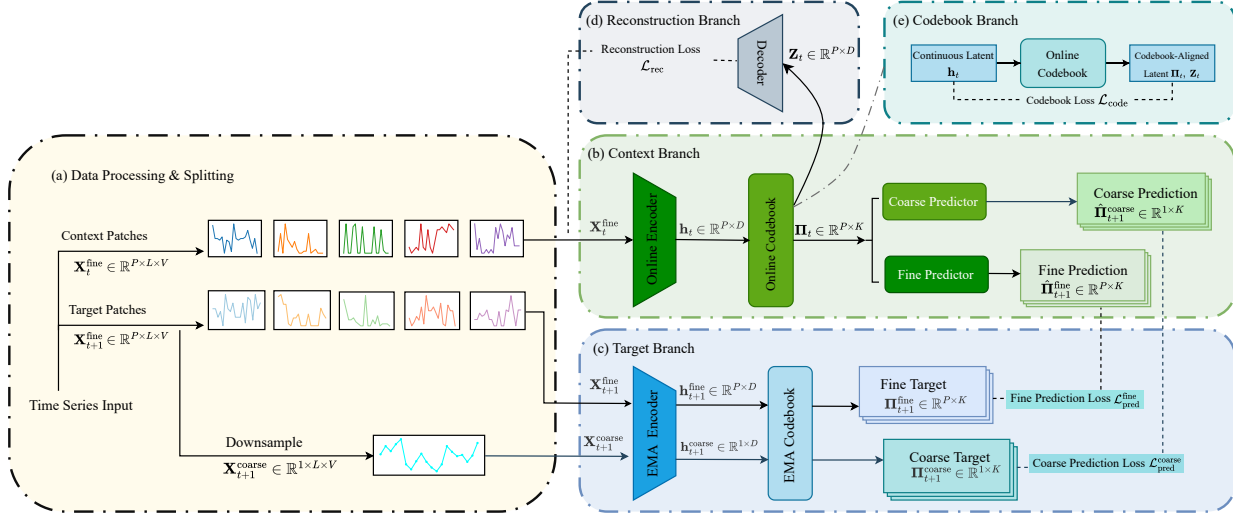


Figure 2. Overview of the SC-JEPA framework for time series representation learning.

However, continuous encoder features often entangle nuisance variations. To mitigate this, we introduce a soft codebook that projects features onto a finite set of prototypes. This discretization not only captures regime-like patterns but also imposes an explicit geometric constraint on the latent space, which, as we detail next, is crucial for ensuring both representation stability and expressiveness.

Codebook. Formally, we implement this projection via a differentiable bottleneck Q that maps continuous encoder features into a finite set of learnable prototypes $\{c_k\}_{k=1}^K \subset \mathbb{R}^D$. For each patch representation $h_{t,i} \in \mathbb{R}^D$, Q outputs a soft code distribution $p_{t,i} \in \Delta^{K-1}$, where $\Delta^{K-1} := \{p \in \mathbb{R}^K : p \succeq 0, \mathbf{1}^\top p = 1\}$. We parameterize $p_{t,i}$ via temperature-scaled cosine similarity:

$$p_{t,i,k} = \frac{\exp\{\langle \bar{h}_{t,i}, \bar{c}_k \rangle / \tau\}}{\sum_{j=1}^K \exp\{\langle \bar{h}_{t,i}, \bar{c}_j \rangle / \tau\}}, \quad (3)$$

where $\bar{h}_{t,i}$ and $\bar{c}_k$ denote ℓ_2 -normalized vectors. We stack these patch-wise distributions to form the window-level code sequence $\Pi_t := [p_{t,1}; \dots; p_{t,P}] \in \mathbb{R}^{P \times K}$, which serves as the predictor input. In parallel, we compute the expected code embedding $z_{t,i} := \sum_{k=1}^K p_{t,i,k} c_k \in \mathbb{R}^D$, which is used by the auxiliary decoder for reconstruction.

Since $p_{t,i} \in \Delta^{K-1}$, the soft embedding $z_{t,i} = \sum_k p_{t,i,k} c_k$ constitutes a convex combination of finitely many prototypes and therefore remains within the bounded convex hull of $\{c_k\}$. This boundedness provides an explicit geometric constraint on the latent space, preventing unbounded feature excursions and improving score stability; a formal **upper bound** is given in Appendix A.3.1. Complementarily, Appendix A.3.2 derives a strictly positive **lower bound** on the batch variance, i.e., $\text{Tr}(\text{Cov}(z)) > 0$, providing a sufficient non-collapse certificate. Together, these bounds ensure

that Π_t provides a compact, regime-level description of the window that is both stable and expressive.

Predictor. With the latent dynamics discretized into a stable code sequence Π_t , the remaining challenge is to forecast the system’s evolution. Recognizing that anomalies often originate from conflicting temporal scales, we employ a dual-branch predictor to explicitly decouple these dynamics. Both branches operate on the same history Π_t but target distinct future resolutions:

- (i) **Fine Predictor (Micro-Dynamics):** To capture high-frequency volatility and local point anomalies, a Transformer maps Π_t to a sequence of fine-grained predictions $\hat{\Pi}_{t+1}^{\text{fine}} \in \mathbb{R}^{P \times K}$. This branch preserves the patch-wise resolution, ensuring that transient disturbances are not smoothed out.
- (ii) **Coarse Predictor (Macro-Dynamics):** To distill low-frequency trends and global distribution shifts, a learnable query token q aggregates the entire history into a single global prediction $\hat{\Pi}_{t+1}^{\text{coarse}} \in \mathbb{R}^{1 \times K}$ via cross-attention:

$$\hat{\Pi}_{t+1}^{\text{coarse}} = \text{CrossAttn}(q, \Pi_t). \quad (4)$$

This mechanism forces the model to abstract away local noise and focus on the trajectory of the system state.

For supervision, the EMA branch generates the corresponding multi-scale targets Π_{t+1}^{fine} and $\Pi_{t+1}^{\text{coarse}}$ using the shared codebook Q .

Decoder. Because prediction alone can encourage overly abstract codes, an auxiliary decoder $\mathcal{D}$ reconstructs the input patches $\mathbf{X}_t^{\text{fine}}$ from the online soft-quantized embeddings $\mathbf{z}_t$ to retain signal-level semantics. The reconstruction is evaluated after inverting RevIN using the cached statistics.

SC-JEPA achieves proactive anomaly prediction via multi-resolution forecasting of regime-level dynamics from the current window. We next formalize the unified training objective in Section 3.4.

3.4. Optimization Objective

We first introduce the notation used to define the three coupled objectives. Let $\mathbf{p}_{t,i} \in \mathbb{R}^K$ denote the online soft code distribution for patch i at window t (Eq. 3), satisfying $\mathbf{p}_{t,i} \geq \mathbf{0}$ and $\mathbf{1}^\top \mathbf{p}_{t,i} = 1$. Let $\mathbf{z}_{t,i} = \sum_{k=1}^K p_{t,i,k} \mathbf{c}_k$ be the corresponding soft-quantized latent. The fine predictor outputs $\hat{\mathbf{p}}_{t+1,i}^{\text{fine}}$ and latent $\hat{\mathbf{z}}_{t+1,i}$ for each patch, while the coarse predictor outputs a single distribution $\hat{\mathbf{p}}_{t+1}^{\text{coarse}}$. All targets $\mathbf{p}_{t+1,i}^{\text{fine}}$, $\mathbf{p}_{t+1}^{\text{coarse}}$ and $\mathbf{z}_{t+1,i}$ are produced by the EMA (target) branch.

Overall objective. We optimize the online encoder, codebook, predictors, and decoder using three terms:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{code}} + \lambda_r \mathcal{L}_{\text{rec}}. \quad (5)$$

With the overall objective in place, we detail each term next.

(A) Predictive Objective ($\mathcal{L}_{\text{pred}}$). We supervise online predictions using EMA targets at both fine (patch) and coarse (window) resolutions, so that the representation is jointly shaped by local precursors and global trends.

$$\mathcal{L}_{\text{pred}} = \underbrace{\lambda_f (\mathcal{L}_{\text{KL}}^{\text{fine}} + \gamma \mathcal{L}_{\text{MSE}}^{\text{fine}})}_{\text{Local Structure}} + \underbrace{\lambda_c \mathcal{L}_{\text{KL}}^{\text{coarse}}}_{\text{Global Dynamics}}. \quad (6)$$

The fine-grained components align code distributions and stabilize latent features, while the coarse objective captures high-level semantics:

$$\mathcal{L}_{\text{KL}}^{\text{fine}} = \sum_i D_{\text{KL}}(\mathbf{p}_{t+1,i}^{\text{fine}} \parallel \hat{\mathbf{p}}_{t+1,i}^{\text{fine}}), \quad (7)$$

$$\mathcal{L}_{\text{MSE}}^{\text{fine}} = \sum_i \|\mathbf{z}_{t+1,i} - \hat{\mathbf{z}}_{t+1,i}\|_2^2, \quad (8)$$

$$\mathcal{L}_{\text{KL}}^{\text{coarse}} = D_{\text{KL}}(\mathbf{p}_{t+1}^{\text{coarse}} \parallel \hat{\mathbf{p}}_{t+1}^{\text{coarse}}). \quad (9)$$

Here, D_{KL} denotes the Kullback–Leibler divergence. λ_f and λ_c balance patch-level supervision with window-level consistency. Since the targets $\mathbf{p}$ are probability vectors produced by the softmax codebook (i.e., $\mathbf{p} \in \Delta^{K-1}$), KL provides a principled objective for distributional alignment in the code space, rather than treating codes as unconstrained Euclidean regressands. In addition, the momentum-updated

EMA branch yields slowly varying targets, which stabilizes self-distillation and makes KL matching better conditioned during training.

(B) Codebook Objective ($\mathcal{L}_{\text{code}}$). A well-behaved bottleneck requires more than defining soft assignments: during training, the code space must stay synchronized with the encoder features, and the assignment distributions must avoid becoming either overly diffuse or collapsed to a few indices. Accordingly, we regularize the soft codebook with (i) prototype–feature alignment and (ii) dual-entropy calibration:

$$\mathcal{L}_{\text{code}} = \underbrace{\lambda_{\text{emb}} \mathcal{L}_{\text{emb}} + \lambda_{\text{com}} \mathcal{L}_{\text{com}}}_{\text{Codebook alignment}} + \underbrace{\lambda_{\text{ent}}^{\text{sample}} \mathcal{L}_{\text{ent}}^{\text{sample}} - \lambda_{\text{ent}}^{\text{batch}} \mathcal{L}_{\text{ent}}^{\text{batch}}}_{\text{Entropy}}. \quad (10)$$

The alignment terms couple continuous encoder features $\mathbf{h}_{t,i}$ with their soft-quantized embeddings $\mathbf{z}_{t,i}$ via a bidirectional objective:

$$\mathcal{L}_{\text{emb}} = \sum_i \|\text{sg}(\mathbf{z}_{t,i}) - \mathbf{h}_{t,i}\|_2^2, \quad (11)$$

$$\mathcal{L}_{\text{com}} = \sum_i \|\mathbf{z}_{t,i} - \text{sg}(\mathbf{h}_{t,i})\|_2^2, \quad (12)$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator.

Meanwhile, we calibrate the assignment distributions through dual-entropy control:

$$\mathcal{L}_{\text{ent}}^{\text{sample}} = \mathbb{E}[H(\mathbf{p})], \quad \mathcal{L}_{\text{ent}}^{\text{batch}} = H(\mathbb{E}[\mathbf{p}]). \quad (13)$$

In particular, minimizing $\mathcal{L}_{\text{ent}}^{\text{sample}}$ sharpens per-sample assignments and encourages decisive code selection, whereas maximizing $\mathcal{L}_{\text{ent}}^{\text{batch}}$ promotes diverse code usage across the dataset, mitigating index collapse. Together, these terms keep the prototype set aligned with the evolving feature space while maintaining stable, informative code distributions for downstream prediction.

(C) Reconstruction Objective ($\mathcal{L}_{\text{rec}}$). To anchor latent representations and prevent collapse, we minimize the reconstruction error between the input patches and their reconstructions (after denormalization):

$$\mathcal{L}_{\text{rec}} = \sum_{i=1}^P \|\hat{\mathbf{X}}_{t,i} - \mathbf{X}_{t,i}\|_2^2. \quad (14)$$

Here $\hat{\mathbf{X}}_{t,i}$ is denormalized using the cached RevIN statistics of window t before computing $\mathcal{L}_{\text{rec}}$.

4. Experiments

4.1. Experimental Setup

Anomaly Prediction Setting. Following prior anomaly prediction frameworks (Jhin et al., 2023; Park et al., 2025;

Table 1. Full results on five benchmark datasets. Baselines are grouped by modeling characteristics. All metrics are reported in percentage (%) as the mean over 5 runs with different random seeds.

Category	Models	MSL				SMAP				SWaT				PSM				SMD			
		F1	AUC	Prec	Rec	F1	AUC	Prec	Rec	F1	AUC	Prec	Rec	F1	AUC	Prec	Rec	F1	AUC	Prec	Rec
Traditional	K-Means (MacQueen, 1967)	20.63	52.17	19.27	31.34	8.74	39.82	10.52	10.16	14.45	62.20	7.99	77.05	42.75	51.37	30.40	78.89	14.79	57.49	35.83	20.64
	DeepSVDD (Ruff et al., 2018)	23.46	53.67	14.78	73.87	22.05	44.07	12.90	81.99	15.28	58.22	12.65	20.00	44.45	49.95	30.31	89.38	9.41	47.31	6.25	19.15
Generalist	LSTM-VAE (Park et al., 2018)	22.42	55.22	20.84	32.43	22.77	52.97	14.35	55.39	54.60	79.91	64.92	50.07	56.73	71.95	49.32	66.86	10.04	56.18	5.78	38.29
	iTransformer (Liu et al., 2024)	27.25	64.61	16.71	75.40	33.00	60.91	22.67	64.32	70.49	82.10	98.17	55.00	54.12	63.09	37.37	98.29	14.20	57.94	8.11	62.34
	PatchTST (Nie et al., 2023)	26.98	60.43	75.00	16.45	30.06	61.62	21.76	49.06	70.64	81.93	94.03	57.58	58.17	75.76	64.68	52.86	11.79	47.76	6.58	57.02
	TS2Vec (Yue et al., 2022)	23.48	64.86	31.45	18.82	32.81	61.48	23.81	52.81	67.00	83.76	80.95	57.17	48.43	72.13	32.39	95.96	14.78	56.28	8.83	50.43
LLM	Qwen2.5-3B (Team, 2024)	21.24	55.66	17.82	26.67	0.34	63.61	1.67	0.19	17.29	56.75	41.82	14.43	47.44	65.65	50.44	46.57	13.00	60.66	8.29	31.06
Hierarchical	TimesNet (Wu et al., 2023)	28.44	59.70	30.31	36.59	27.78	57.01	17.98	62.81	67.20	83.24	90.87	54.42	52.64	56.12	35.76	99.71	12.51	56.33	7.64	35.96
	MICN (Wang et al., 2023)	20.00	60.18	11.56	75.29	20.81	51.16	13.75	55.58	48.31	79.38	77.99	35.33	48.61	54.97	34.64	82.09	12.05	52.21	7.72	28.72
	Pathformer (Chen et al., 2024)	21.69	61.86	13.14	68.24	23.58	58.67	14.10	72.12	14.90	59.87	9.20	40.98	51.77	54.41	35.04	99.10	14.32	56.49	9.41	36.60
Anomaly Prediction	PAD (Jhin et al., 2023)	21.67	55.97	12.19	97.78	26.92	59.79	18.15	55.00	69.75	83.71	81.54	62.79	57.82	73.84	57.54	58.57	12.52	58.53	6.71	93.40
	A2P (Park et al., 2025)	23.10	59.43	13.92	74.12	21.68	60.37	15.88	38.85	70.18	81.92	99.10	54.33	50.71	57.01	37.16	80.00	15.21	58.87	12.23	22.13
	FCM (Zhao et al., 2024)	24.57	61.09	14.25	90.00	30.60	61.85	25.25	41.41	69.09	81.66	94.25	54.83	52.34	57.25	37.00	89.85	19.25	60.44	15.59	28.94
JEPAs	TS-JEPA (Ennadir et al., 2025)	25.49	60.33	17.10	56.84	26.57	57.38	18.23	49.73	71.95	80.33	84.32	62.76	53.32	66.10	36.95	96.29	7.49	49.53	19.11	20.43
	SC-JEPA (Ours)	33.58	66.08	35.87	40.80	33.64	65.41	24.24	56.02	72.89	84.95	98.00	58.05	61.61	77.85	55.01	72.00	20.82	62.29	18.18	31.49

Zhao et al., 2024), we partition each time series into non-overlapping windows of length 100. For each sample, the current window $\mathbf{X}_t$ is used as the model input, while the next window $\mathbf{X}_{t+1}$ is used to define the supervision label Y_{t+1} . If $\mathbf{X}_{t+1}$ contains at least one anomalous time point, we assign $Y_{t+1} = 1$; otherwise, we assign $Y_{t+1} = 0$. The model is then trained to use only $\mathbf{X}_t$ to predict Y_{t+1} , namely whether the next non-overlapping window will contain an anomaly. Under this formulation, $\mathbf{X}_t$ serves as the precursor context, and the task evaluates whether the model can identify early warning signals before anomalies fully emerge. We report window-level Precision, Recall, F1, and AUC.

Datasets and Baselines. We evaluate on five widely-used multivariate anomaly detection benchmarks spanning spacecraft telemetry, industrial control, and cloud services. Mars Science Laboratory (MSL) and Soil Moisture Active Passive (SMAP) are NASA spacecraft telemetry datasets (Hundman et al., 2018). Secure Water Treatment (SWaT) is an industrial control system (ICS) water-treatment testbed with attack scenarios (Goh et al., 2017). Pooled Server Metrics (PSM) is a public server monitoring dataset collected from eBay production machines (Su et al., 2019). Server Machine Dataset (SMD) is a large-scale multivariate time-series benchmark collected from multiple server machines (Su et al., 2019). To evaluate SC-JEPA, we benchmark against six categories of baselines: (1) traditional methods: K-Means (MacQueen, 1967), DeepSVDD (Ruff et al., 2018); (2) generalist sequence models: LSTM-VAE (Park et al., 2018), iTransformer (Liu et al., 2024), PatchTST (Nie et al., 2023), TS2Vec (Yue et al., 2022); (3) LLM: Qwen2.5-3B (Team, 2024); (4) hierarchical models: TimesNet (Wu et al., 2023), MICN (Wang et al., 2023), Pathformer (Chen et al., 2024); (5) anomaly prediction methods: PAD (Jhin et al., 2023), A2P (Park et al., 2025), FCM (Zhao et al., 2024); and (6) JEPA-based models: TS-JEPA (Ennadir et al., 2025).

Training & Evaluation Protocol. We convert each multivariate stream into non-overlapping context-target pairs with stride 100, yielding windows with context length $T_c = 100$ and target length $T_t = 100$. Each window is tokenized into $P = 5$ non-overlapping patches of length $L = 20$. We use a two-phase pipeline. In self-supervised pre-training, we train on the official training set without labels and use a 9:1 train-validation split for model selection and early stopping. For downstream training on the official test set, we adopt a 6:2:2 chronological split: we train the classifier on the training split, select the detection threshold on the validation split, and then fix it for final evaluation on the held-out test split. We report window-level Precision, Recall, F1, and AUC. Full implementation details, hyperparameters, and the downstream protocol are provided in Appendix B.1 and Appendix B.3.

4.2. Results

Main results. Table 1 reports the mean anomaly prediction results on five benchmarks using the same data splits and five random seeds. Full results are available in Appendix C. Under this setting, SC-JEPA achieves the top AUC on all datasets, suggesting consistently strong ranking quality for upcoming abnormal windows under our precursor-based definition. Moreover, SC-JEPA attains best F1 performance across benchmarks, indicating that its ranking quality can translate into effective decisions under the shared threshold-selection procedure.

The trends of competing baselines are broadly consistent with their modeling assumptions. Classical methods such as K-Means and DeepSVDD rely on static similarity or compactness in a feature space, which can be less sensitive when precursor windows deviate only mildly from normal patterns. Reconstruction-based models such as LSTM-VAE may also be less aligned with a precursor setting, since

Table 2. In-domain and cross-domain generalization results. Mean F1 and AUC (%) are reported over 5 runs. ID: in-domain; CD: cross-domain; Δ : relative change from ID to CD.

Models	Settings	MSL		SMAP		SWaT		PSM	
		F1	AUC	F1	AUC	F1	AUC	F1	AUC
PatchTST	ID	26.98	60.43	30.06	61.62	70.64	81.93	58.17	75.76
	CD	27.08	60.80	26.21	58.77	54.34	68.76	50.07	53.72
	Δ	+0.37%	+0.61%	-12.81%	-4.63%	-23.07%	-16.07%	-13.92%	-29.09%
TS2Vec	ID	23.48	64.86	32.81	61.48	67.00	83.76	48.43	72.13
	CD	22.87	62.00	26.99	57.54	65.08	82.91	48.15	56.22
	Δ	-2.60%	-4.41%	-17.74%	-6.41%	-2.87%	-1.01%	-0.58%	-22.06%
SC-JEPA	ID	33.58	66.08	33.64	65.41	72.89	84.95	61.61	77.85
	CD	33.48	67.13	33.81	68.88	71.10	82.96	56.87	66.80
	Δ	-0.30%	+1.59%	+0.51%	+5.30%	-2.46%	-2.34%	-7.69%	-14.19%

reconstruction error often becomes more distinctive after abnormal behavior has developed. In addition, a set of advanced time-series backbones, including TimesNet, TS2Vec, PatchTST, and iTransformer, provide strong sequence representations and capture long-range dependencies. In our anomaly prediction setting, however, these general-purpose backbones are not directly optimized for highlighting weak early-stage transitions toward future abnormalities, so their scores can be less separable when the precursor signal is subtle and noisy. Likewise, the inferior performance of TS-JEPA highlights the limitations of a pure continuous predictive model; lacking both a discrete codebook and multi-resolution modeling, it suffers from optimization instability and fails to capture diverse precursor dynamics. Finally, while PAD is explicitly designed for anomaly prediction, it relies on a single-scale objective that restricts its ability to decouple complex latent evolution from stochastic noise.

4.3. Generality

A practical time-series model should remain effective under distribution shifts, where the pre-training data may differ substantially from the deployment environment. To assess such transferability, Table 2 reports generality results under two pre-training settings. In-domain pre-training uses the official training set of the target dataset, while cross-domain pre-training uses the union of the other three datasets, with the target dataset fully excluded during pre-training. In both cases, we keep the downstream evaluation protocol identical on the target dataset, so the only difference comes from the source of pre-training data. We select TS2Vec and PatchTST for this generality study since they perform strongly in our main table under our setting, and thus provide meaningful SOTA references for evaluating transferability.

With this setup, Table 2 shows that cross-domain transfer is generally more challenging, and performance can decrease when the pre-training data comes from different datasets, which is consistent with the presence of domain shift. Notably, SC-JEPA continues to perform competitively across benchmarks. We observe that its AUC remains comparable to the in-domain setting on several datasets and can improve modestly in some cases, suggesting that the learned

Table 3. Ablation results on five benchmark datasets (%).

Model Variant	MSL		SMAP		SWaT		PSM		SMD	
Metric	F1	AUC	F1	AUC	F1	AUC	F1	AUC	F1	AUC
w/o KL Divergence	29.01	53.90	28.71	60.19	67.33	79.95	54.93	70.83	12.63	53.61
w/o Reconstruction	25.96	52.68	25.15	52.80	14.77	53.30	53.03	69.84	10.18	50.44
w/o Predictive Objective	28.88	53.17	27.05	59.81	59.47	78.60	58.89	70.17	10.04	50.24
w/o Codebook Loss	31.62	58.93	31.53	62.06	72.64	82.63	60.83	75.85	12.82	57.36
w/o Codebook Module	21.82	43.02	21.69	51.00	11.51	50.00	53.03	46.61	12.45	50.00
w/o Temporal Downsampling	28.77	63.16	30.39	61.34	71.62	83.60	60.79	71.72	15.50	60.93
Full Model	33.58	66.08	33.64	65.41	72.89	84.95	61.61	77.85	20.82	62.29

scores preserve useful ranking information even when the target dataset is excluded during pre-training. On more challenging data such as PSM, all methods experience a clear drop, yet SC-JEPA retains relatively strong performance compared to the evaluated backbones. Overall, these results provide additional evidence that the model transfers reasonably well across datasets, particularly when assessed through threshold-free ranking metrics.

4.4. Model Analysis

4.4.1. ABLATION STUDY

Since SC-JEPA integrates multiple mechanisms, it is important to attribute the observed gains to the intended design choices rather than a single component. We therefore conduct a systematic ablation study by selectively removing modules, as summarized in Table 3. We observe that the predictive objective is the primary engine for capturing temporal dependencies; removing it causes a significant drop, and the KL term proves superior to MSE alone by enforcing precise distributional alignment. Furthermore, the results validate that multi-resolution modeling is essential for outperforming baselines, as it enables the capture of coarse-grained precursors that single-scale models miss. Conversely, reconstruction serves as a non-negotiable anchor; its removal triggers model collapse, confirming its role in preventing latent degeneration.

Among all variants, the most consequential one is removing the soft codebook bottleneck (w/o Codebook Module): performance collapses to near-random, while removing only the auxiliary regularizers yields a much smaller drop. It suggests that beyond providing discrete, prototype-anchored regime codes, the bottleneck may also play a critical role in optimization stability. This naturally raises a question: what stabilizing effect does the bottleneck introduce that the auxiliary losses alone do not?

The ablations offer an empirical explanation consistent with our formulation. With the codebook in place, self-distillation operates as KL matching between probability codes on the simplex (Eq. 3), so the predictor tracks a bounded distributional trajectory instead of performing unconstrained regression in $\mathbb{R}^D$. In conjunction with slowly varying EMA targets, this bounded code space appears to provide a more stable reference for self-distillation (Zhou

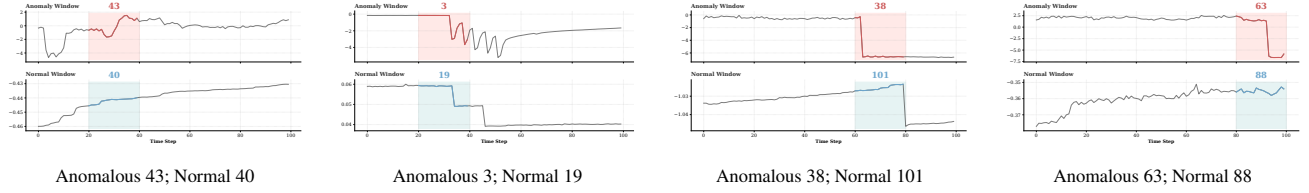


Figure 3. Window-level comparisons of dominant latent codes between anomalous and normal windows on the PSM dataset. Shaded regions indicate patch-level segments where the dominant code is selected.

Table 4. Variance ratio comparison on three datasets. Lower values indicate less representation collapse; gray text shows the relative decrease versus the variant without codebook.

Dataset	Method	Top-1 ↓	Top-5 ↓	Top-10 ↓
PSM	without codebook	0.5334	1.0000	1.0000
	with codebook (ours)	0.1765 (↓66.9%)	0.4347 (↓56.5%)	0.6778 (↓32.2%)
MSL	without codebook	0.3292	0.9088	0.9154
	with codebook (ours)	0.1804 (↓45.2%)	0.5194 (↓42.8%)	0.6607 (↓27.8%)
SWaT	without codebook	0.3251	0.9271	0.9344
	with codebook (ours)	0.1143 (↓64.8%)	0.3190 (↓65.6%)	0.4041 (↓56.8%)

et al., 2022). Once we remove the bottleneck, this constrained code space disappears, so KL alignment loses its natural domain, and training degenerates into an ill-conditioned matching problem, consistent with the observed near-collapse. By contrast, removing only the auxiliary codebook losses causes only a mild drop for w/o Codebook Loss. These auxiliary losses mainly serve as an outperformance mechanism that further boosts an already strong model.

4.4.2. VISUALIZATION OF LEARNED REGIMES

In the Introduction, we hypothesized that a discrete bottleneck would sharpen the distinction between normal dynamics and structural anomalies. Furthermore, in Section 3.3, we designed the codebook $\mathcal{Q}$ specifically to capture regime-like patterns via finite prototypes $\{\mathbf{c}_k\}$. Here, we empirically verify whether the learned representations align with these methodological motivations.

First, we examine the activation statistics of the codebook. Figure 4 reports a quantitative study on the PSM dataset, ranking codes by the absolute frequency gap between normal and anomalous windows. We observe that a small subset of codes exhibits pronounced activation gaps, suggesting that the model naturally learns to partition the latent space into condition-dependent regions without explicit supervision.

To further validate this structure qualitatively, Figure 3 maps the most discriminative codes back to their original time-series patches. We observe that, within the same temporal context, distinct codes consistently correspond to distinct physical behaviors. This alignment confirms that the discrete formulation effectively disentangles diverse local dy-

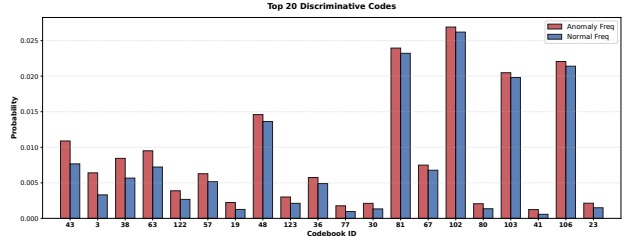


Figure 4. Histogram comparing the occurrence probability of latent codes in anomalous versus normal windows on the PSM test set. The codes are sorted by their anomaly-normal probability difference (from largest to smallest). PSM is used because it provides a large test set with reliable window-level anomaly labels.

namics into a finite set of identifiable regimes. Consequently, the codebook successfully distinguishes precursor-like patterns from normal dynamics, providing a discriminative structural basis for anomaly prediction even under distribution shift.

5. Conclusion

This work introduces SC-JEPA, advancing anomaly prediction from reactive detection to proactive world modeling. By synergizing multi-resolution views with a soft-quantized bottleneck, our architecture effectively captures scale-variant precursors while mitigating the representation collapse often observed in continuous self-supervision. Our empirical and theoretical analysis reveals that the discrete bottleneck serves as a critical regularizer, enabling stable optimization without explicit negative sampling. Extensive experiments confirm that SC-JEPA achieves state-of-the-art performance in early-warning scenarios, suggesting that discrete latent dynamics offer a robust foundation for future research in reliable autonomous systems.

Impact Statement

This work advances machine learning methods for early warning from multivariate time series. By improving the stability of latent predictive learning and modeling precursor patterns across temporal scales, SC-JEPA may support earlier identification of emerging failures in industrial, infrastructure, and computing systems. Such capabilities could help reduce downtime, operational cost, and safety risks

when deployed with appropriate human oversight. However, inaccurate predictions may lead to missed failures or unnecessary interventions, and performance may degrade under distribution shifts or in domains not represented by the training data. Therefore, the method should be carefully validated for each application and used as decision support rather than as a fully autonomous safety mechanism.

References

- Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., and Ballas, N. Self-supervised learning from images with a Joint-Embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15619–15629, 2023.
- Bardes, A., Ponce, J., and LeCun, Y. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., and Ballas, N. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- Chen, X. and He, K. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15750–15758, 2021.
- Chen, Y., Zhang, Z., Wu, H., Wang, L., Zhao, S., and Long, M. Pathformer: Multi-scale transformers with adaptive pathways for time series forecasting. In *International Conference on Learning Representations*, 2024.
- Cover, T. M. and Thomas, J. A. *Elements of Information Theory*. Wiley-Interscience, Hoboken, N.J., 2nd edition, 2006.
- Ennadir, S., Golkar, S., and Sarra, L. Joint embeddings go temporal. *arXiv preprint arXiv:2509.25449*, 2025.
- Goh, J., Adepu, S., Junejo, K. N., and Mathur, A. A dataset to support research in the design of secure water treatment systems. In Havarneanu, G., Setola, R., Nassopoulos, H., and Wolthusen, S. (eds.), *Critical Information Infrastructures Security*, pp. 88–99, Cham, 2017. Springer International Publishing.
- Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., Doersch, C., Pires, B. A., Guo, Z., Azar, M. G., et al. Bootstrap your own latent—a new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, volume 33, pp. 21271–21284, 2020.
- Hundman, K., Constantinou, V., Laporte, C., Colwell, I., and Soderstrom, T. Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 387–395, 2018.
- Jhin, S. Y., Lee, J., and Park, N. Precursor-of-anomaly detection for irregular time series. In *Proceedings of the 29th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 917–929, 2023.
- Jing, L., Vincent, P., LeCun, Y., and Tian, Y. Understanding dimensional collapse in contrastive self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- Kim, T., Kim, J., Tae, Y., Park, C., Choi, J.-H., and Choo, J. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations*, 2022.
- Lee, D., Malacarne, S., and Aune, E. Vector quantized time series generation with a bidirectional prior model. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, 2023.
- Lee, D., Malacarne, S., and Aune, E. Explainable time series anomaly detection using masked latent generative modeling. *Pattern Recognition*, 156:110826, 2024.
- Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. Feature pyramid networks for object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2117–2125, 2017.
- Liu, M., Zeng, A., Chen, M., Xu, Z., Lai, Q., Ma, L., and Xu, Q. SCINet: Time series modeling and forecasting with sample convolution and interaction. *Advances in Neural Information Processing Systems*, 35:5816–5828, 2022a.
- Liu, S., Yu, H., Liao, C., Li, J., Lin, W., Liu, A. X., and Dustdar, S. Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. In *International Conference on Learning Representations*, 2022b.
- Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., and Long, M. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations*, 2024.
- MacQueen, J. Some methods for classification and analysis of multivariate observations. In *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, Volume 1*, pp. 281–297. University of California Press, 1967.

- Nie, Y., Nguyen, N. H., Sinthong, P., and Kalagnanam, J. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.
- Park, D., Hoshi, Y., and Kemp, C. C. A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder. *IEEE Robotics and Automation Letters*, 3(3):1544–1551, 2018.
- Park, M.-Y., Lee, W.-J., Kim, S. T., and Park, G.-M. When will it fail?: Anomaly to prompt for forecasting future anomalies in time series. *arXiv preprint arXiv:2506.23596*, 2025.
- Ruff, L., Vandermeulen, R., Goernitz, N., Deecke, L., Siddiqui, S. A., Binder, A., Müller, E., and Kloft, M. Deep one-class classification. In *International Conference on Machine Learning*, pp. 4393–4402, 2018.
- Su, Y., Zhao, Y., Niu, C., Liu, R., Sun, W., and Pei, D. Robust anomaly detection for multivariate time series through stochastic recurrent neural networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- Team, Q. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Tian, Y., Chen, X., and Ganguli, S. Understanding self-supervised learning dynamics without contrastive pairs. In *International Conference on Machine Learning*, pp. 10268–10278. PMLR, 2021.
- Van Den Oord, A., Vinyals, O., et al. Neural discrete representation learning. *Advances in Neural Information Processing Systems*, 30, 2017.
- Wang, H., Peng, J., Huang, F., Wang, J., Chen, J., and Chu, Y. Micn: Multi-scale local and global context modeling for long-term series forecasting. In *International Conference on Learning Representations*, 2023.
- Wang, S., Wu, H., Shi, X., Hu, T., Luo, H., Ma, L., Zhang, J. Y., and Zhou, J. TimeMixer: Decomposable multi-scale mixing for time series forecasting. In *International Conference on Learning Representations*, 2024.
- Wen, Y., Ma, T., Weng, L., Nguyen, L., and Julius, A. A. Abstracted shapes as tokens-a generalizable and interpretable model for time-series classification. *Advances in Neural Information Processing Systems*, 37:92246–92272, 2025.
- Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., and Long, M. TimesNet: Temporal 2d-variation modeling for general time series analysis. In *International Conference on Learning Representations*, 2023.
- Xu, J., Wu, H., Wang, J., and Long, M. Anomaly transformer: Time series anomaly detection with association discrepancy. In *International Conference on Learning Representations*, 2022.
- Yue, Z., Wang, Y., Duan, J., Yang, T., Huang, C., Tong, Y., and Xu, B. TS2Vec: Towards universal representation of time series. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 8980–8987, 2022.
- Zhao, S., Wang, W., Xu, H., Yu, Z., Wen, Q., Wang, G., and Pang, G. Abnormality forecasting: Time series anomaly prediction via future context modeling. *arXiv preprint arXiv:2410.12206*, 2024.
- Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., and Zhang, W. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 11106–11115, 2021.
- Zhou, J., Wei, C., Wang, H., Shen, W., Xie, C., Yuille, A., and Kong, T. Image BERT pre-training with online tokenizer. In *International Conference on Learning Representations*, 2022.

Appendix Contents

A. Methodological Details	12
A.1 Notations	12
A.2 Time-Axis Downsampling	12
A.3 Theoretical Analysis of Latent Stability	12
A.3.1 Stability Upper Bound	12
A.3.2 Non-Collapse Lower Bound	14
B. Reproducibility Details	16
B.1 Implementation Details	16
B.2 SC-JEPA Pre-training Algorithm	17
B.3 Downstream Anomaly Prediction Protocol	17
C. Experimental Results	18
C.1 Main Benchmark	18
C.2 Generality	19
C.3 Ablation Studies	19
D. Visualizations and Qualitative Analysis	20
D.1 Framework	20
D.2 Inference Efficiency	21

A. Methodological Details

A.1. Notations

Table 5. Categorized notations.

Category	Symbol	Description	Category	Symbol	Description
Dimensions	T, V	Time series size ($T \times V$)	Data Inputs	$\mathbf{x}$	Input time series
	T_w	Window length		$\hat{\mathbf{W}}_t$	Normalized window
	N	Number of windows		$\mathbf{X}_t^{\text{fine}}$	Fine view (patches)
	P	Num. patches ($T_w = P \cdot L$)		$\mathbf{X}_t^{\text{coarse}}$	Coarse view (downsampled)
	L	Patch length	Representations	$\mathbf{h}_t$	Encoder latent rep.
	K	Codebook size		$\mathbf{c}_k$	Learnable prototype
	D	Embedding dimension		$\mathbf{p}_{t,i}$	Code-assignment prob.
Hyperparams	τ	Soft-max temperature		$\mathbf{\Pi}_t$	Code sequence
	$\lambda_{\dots}$	Loss balancing weights		$\mathbf{z}_{t,i}$	Soft-quantized embedding
	γ	MSE local weight		$\mathbf{q}$	Coarse query token
Architecture	$\mathcal{E}_\theta$	Online encoder	Outputs	$\hat{\mathbf{\Pi}}^{\text{fine}}$	Predicted fine code dist.
	$\mathcal{E}_\xi$	EMA encoder		$\hat{\mathbf{\Pi}}^{\text{coarse}}$	Predicted coarse code dist.
	$\mathcal{Q}$	Codebook bottleneck		$\hat{\mathbf{z}}_{t,i}$	Predicted embedding
	$\mathcal{D}$	Reconstruction decoder		$\hat{\mathbf{X}}_{t,i}$	Reconstructed patch
	sg	Stop-gradient operator	Objectives	$\mathcal{L}_{\text{pred/code}}$	Predictive / Code losses
				$\mathcal{L}_{\text{rec/total}}$	Recon. / Total losses

A.2. Time-Axis Downsampling

DownAvg operator. Given a normalized window $\hat{\mathbf{W}}_t \in \mathbb{R}^{T_w \times V}$ with $T_w = P \cdot L$, we define $\text{DownAvg}_P(\cdot)$ as a time-axis downsampling operator that averages every P consecutive time points (stride P) for each variable:

$$\mathbf{X}_t^{\text{coarse}} = \text{DownAvg}_P(\hat{\mathbf{W}}_t) \in \mathbb{R}^{1 \times L \times V}. \quad (15)$$

Equivalently, for within-patch index $\ell \in [L]$ and variable $v \in [V]$:

$$[\mathbf{X}_t^{\text{coarse}}]_{1,\ell,v} = \frac{1}{P} \sum_{j=1}^P \hat{\mathbf{W}}_t[(\ell-1)P + j, v]. \quad (16)$$

A.3. Theoretical Analysis of Latent Stability

In this subsection, we provide the formal proofs for the stability properties of the SC-JEPA latent space. We demonstrate how the soft codebook acts as a geometric constraint that explicitly bounds representation drift. We provide two complementary guarantees: (i) an upper bound on representation drift (stability), and (ii) a lower bound that rules out the trivial constant solution (non-collapse).

A.3.1. STABILITY UPPER BOUND

This part proves an explicit upper bound on representation drift in embedding space. The argument follows a short chain: we first fix a bounded latent geometry via the soft codebook, then establish a probability-to-geometry bridge ($\ell_1 \rightarrow \ell_2$), and finally combine this bridge with EMA smoothness and predictive alignment to obtain Theorem A.3.

Preliminaries. Continuous forecasting models can exhibit collapse or explosion, making the latent geometry unstable. The soft codebook eliminates this degree of freedom by restricting every soft-quantized embedding to the convex hull of a

finite prototype set. In particular, the radius constant M defined below will be the only scale factor that propagates into the final drift bound.

Definition A.1 (Soft Code Map). Let $K \geq 2$ be the number of discrete prototypes and $D \geq 1$ be the embedding dimension. Let $\mathbf{C} \in \mathbb{R}^{K \times D}$ be the codebook matrix where the k -th **row** corresponds to the prototype $\mathbf{c}_k$. We define the probability simplex $\Delta^{K-1} := \{\mathbf{p} \in \mathbb{R}^K : \mathbf{p} \succeq \mathbf{0}, \mathbf{1}^\top \mathbf{p} = 1\}$.

The **soft-quantization map** $z : \Delta^{K-1} \rightarrow \mathbb{R}^D$ is defined as the expected embedding:

$$z(\mathbf{p}) := \mathbf{C}^\top \mathbf{p} = \sum_{k=1}^K p_k \mathbf{c}_k. \quad (17)$$

We further define the **geometric radius** M as the maximum Euclidean norm of any prototype...

Lipschitz Bound. Training is performed in probability space (via KL divergence), whereas drift and anomaly scores are measured in embedding space. We therefore need a simple link that converts discrepancies between assignment distributions into Euclidean changes after soft quantization. Lemma A.2 provides exactly this bridge, with the codebook radius M as the controlling constant.

Lemma A.2 ($\ell_1 \rightarrow \ell_2$ Lipschitzness). *For any two assignment distributions $\mathbf{p}, \mathbf{q} \in \Delta^{K-1}$, the Euclidean distance between their soft-quantized embeddings is bounded by their ℓ_1 distance, scaled by the codebook radius M :*

$$\|z(\mathbf{p}) - z(\mathbf{q})\|_2 \leq M \|\mathbf{p} - \mathbf{q}\|_1. \quad (18)$$

Proof. We expand the Euclidean distance and apply the triangle inequality.

$$\begin{aligned} \|z(\mathbf{p}) - z(\mathbf{q})\|_2 &= \left\| \sum_{k=1}^K p_k \mathbf{c}_k - \sum_{k=1}^K q_k \mathbf{c}_k \right\|_2 \\ &= \left\| \sum_{k=1}^K (p_k - q_k) \mathbf{c}_k \right\|_2 \\ &\leq \sum_{k=1}^K |p_k - q_k| \cdot \|\mathbf{c}_k\|_2. \end{aligned}$$

Since $\|\mathbf{c}_k\|_2 \leq M$ for all k , we have:

$$\sum_{k=1}^K |p_k - q_k| \cdot \|\mathbf{c}_k\|_2 \leq M \sum_{k=1}^K |p_k - q_k| = M \|\mathbf{p} - \mathbf{q}\|_1.$$

□

With this bridge in place, the remaining step is to control the probability-space differences themselves. Predictive alignment bounds the deviation between targets and predictions at each time step, while EMA smoothness bounds how fast the target moves. Combining these two controls yields the following stability bound.

Theorem A.3 (Stability Bound). *Let $\{\hat{\mathbf{p}}_t\}$ and $\{\mathbf{p}_t\}$ be the sequences of predicted and target (EMA) distributions, respectively, and let $\hat{\mathbf{z}}_t = z(\hat{\mathbf{p}}_t)$. Assume the following conditions hold for time steps t and $t+1$:*

1. **Predictive Alignment:** $D_{\text{KL}}(\mathbf{p}_t \| \hat{\mathbf{p}}_t) \leq \varepsilon_t$ and $D_{\text{KL}}(\mathbf{p}_{t+1} \| \hat{\mathbf{p}}_{t+1}) \leq \varepsilon_{t+1}$.
2. **Target Smoothness:** $\|\mathbf{p}_{t+1} - \mathbf{p}_t\|_1 \leq \delta_t$.

Then, the representation drift $\|\hat{\mathbf{z}}_{t+1} - \hat{\mathbf{z}}_t\|_2$ is bounded by:

$$\|\hat{\mathbf{z}}_{t+1} - \hat{\mathbf{z}}_t\|_2 \leq M \left(\sqrt{2\varepsilon_{t+1}} + \delta_t + \sqrt{2\varepsilon_t} \right). \quad (19)$$

Proof. First, we recall **Pinsker’s Inequality** (Cover & Thomas, 2006), which provides a bound on the ℓ_1 distance between two distributions in terms of their KL divergence:

$$\|\mathbf{p} - \mathbf{q}\|_1 \leq \sqrt{2D_{\text{KL}}(\mathbf{p} \parallel \mathbf{q})}.$$

We now decompose the drift term $\|\hat{\mathbf{z}}_{t+1} - \hat{\mathbf{z}}_t\|_2$ using the triangle inequality by introducing the intermediate target embeddings $\mathbf{z}_{t+1}$ and $\mathbf{z}_t$:

$$\begin{aligned} \|\hat{\mathbf{z}}_{t+1} - \hat{\mathbf{z}}_t\|_2 &= \|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1} + \mathbf{z}_{t+1} - \mathbf{z}_t + \mathbf{z}_t - \hat{\mathbf{z}}_t\|_2 \\ &\leq \underbrace{\|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\|_2}_{(A)} + \underbrace{\|\mathbf{z}_{t+1} - \mathbf{z}_t\|_2}_{(B)} + \underbrace{\|\mathbf{z}_t - \hat{\mathbf{z}}_t\|_2}_{(C)}. \end{aligned}$$

Bounding (A) and (C): Using Lemma A.2 and Pinsker’s Inequality:

$$\|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\|_2 \leq M\|\hat{\mathbf{p}}_{t+1} - \mathbf{p}_{t+1}\|_1 \leq M\sqrt{2\varepsilon_{t+1}}.$$

By symmetry, $\|\mathbf{z}_t - \hat{\mathbf{z}}_t\|_2 \leq M\sqrt{2\varepsilon_t}$.

Bounding (B): Using Lemma A.2 and the smoothness assumption:

$$\|\mathbf{z}_{t+1} - \mathbf{z}_t\|_2 \leq M\|\mathbf{p}_{t+1} - \mathbf{p}_t\|_1 \leq M\delta_t.$$

Summing these terms yields the final bound. $\square$

Interpretation. This theorem implies that the sensitivity of our anomaly score is strictly controlled by the codebook radius M . Unlike unconstrained continuous models where gradients can cause unbounded output shifts (effectively $M \rightarrow \infty$), SC-JEPA guarantees that any significant representation drift arises from either a true shift in the target signal (δ_t) or a high prediction error (ε). This effectively filters out false positives caused by trivial numerical perturbations.

A.3.2. NON-COLLAPSE LOWER BOUND

Goal. We provide a simple *non-collapse certificate* for the soft codebook. Namely, if a mini-batch exhibits (i) non-trivial multi-code usage, (ii) sufficiently sharp assignments, and (iii) non-negligible separation between two frequently used prototypes, then the soft-quantized embeddings $\mathbf{z}_i = \mathbf{C}^\top \mathbf{p}_i$ have strictly positive batch variance, i.e., $\text{Tr}(\text{Cov}(\{\mathbf{z}_i\})) > 0$.

Setup. Consider a batch of assignment probabilities $\{\mathbf{p}_i\}_{i=1}^B \subset \Delta^{K-1}$ and their average $\bar{\mathbf{p}} := \frac{1}{B} \sum_{i=1}^B \mathbf{p}_i$. Let $\mathbf{z}_i := \mathbf{C}^\top \mathbf{p}_i \in \mathbb{R}^D$, where $\mathbf{C} \in \mathbb{R}^{K \times D}$ stacks prototypes as rows.

To preclude the constant-code corner case, we assume three mild properties. Crucially, these are not arbitrary assumptions but are explicitly enforced by the optimization objectives defined in Section 3.4:

- **Batch Entropy:** $\mathcal{L}_{\text{ent}}^{\text{batch}}$ maximizes $H(\bar{\mathbf{p}})$, enforcing Assumption A.4.
- **Sharpness:** $\mathcal{L}_{\text{ent}}^{\text{sample}}$ minimizes individual entropy, driving $\mathbf{p}_i$ toward one-hot vectors (Assumption A.5).
- **Separation:** $\mathcal{L}_{\text{com}}$ and EMA updates prevent prototype collapse (Assumption A.7).

Assumption A.4 (Multi-code usage / non-trivial batch marginal). The batch marginal $\bar{\mathbf{p}}$ is not concentrated on a single index. Concretely,

$$H(\bar{\mathbf{p}}) \geq \eta \quad \text{for some } \eta > 0. \quad (20)$$

Assumption A.5 (Assignment sharpness). For each sample, let $k(i) = \arg \max_k p_{i,k}$. There exists $\varepsilon \in [0, 1)$ such that

$$\|\mathbf{p}_i - \mathbf{e}_{k(i)}\|_1 \leq \varepsilon \quad \text{for all } i. \quad (21)$$

Remark A.6. The deviation ε is strictly controlled by the softmax temperature τ . As $\tau \rightarrow 0$, $\mathbf{p}_i$ approaches a hard assignment, ensuring $\varepsilon \rightarrow 0$.

Assumption A.7 (Separation for the two dominant codes). Define

$$\beta := \frac{1 - \rho^*}{K - 1}, \quad (22)$$

where $\rho^* \in (1/K, 1)$ is the unique root of the equation $\mathcal{H}_b(\rho) + (1 - \rho) \log(K - 1) = \eta$. Let $m = \arg \max_k \bar{p}_k$ and let $r \neq m$ be any index satisfying $\bar{p}_r \geq \beta$ (guaranteed by Lemma A.8). Assume their prototypes satisfy

$$\|\mathbf{c}_m - \mathbf{c}_r\|_2 \geq \Delta_c \quad \text{for some } \Delta_c > 0. \quad (23)$$

To drive a variance lower bound, we first (Lemma A.8) extract two non-negligible codes from entropy, then (Lemma A.9) convert their marginal mass into two dominant-code groups under sharp assignments.

Lemma A.8 (Entropy lower bound implies at least two active codes). *Let $m = \arg \max_k \bar{p}_k$ and $\bar{p}_{\max} = \bar{p}_m$. If $H(\bar{\mathbf{p}}) \geq \eta$, then $\bar{p}_{\max} \leq \rho^*$, where ρ^* is defined in Assumption A.7. Moreover, there exists an index $r \neq m$ such that*

$$\bar{p}_r \geq \beta := \frac{1 - \rho^*}{K - 1}. \quad (24)$$

Proof. Consider the function $\phi(x) := \mathcal{H}_b(x) + (1 - x) \log(K - 1)$, which represents the maximum entropy achievable by a distribution with maximum probability mass x . It is well-known (from Fano's inequality analysis) that $\phi(x)$ is strictly decreasing for $x \in [1/K, 1]$. Since $H(\bar{\mathbf{p}}) \geq \eta$ and $H(\bar{\mathbf{p}}) \leq \phi(\bar{p}_{\max})$, we have $\phi(\bar{p}_{\max}) \geq \eta = \phi(\rho^*)$. By the monotonicity of ϕ , this implies $\bar{p}_{\max} \leq \rho^*$. Consequently, the remaining mass is at least $1 - \rho^*$. By the pigeonhole principle, at least one non-dominant index r must satisfy $\bar{p}_r \geq (1 - \rho^*)/(K - 1) = \beta$. $\square$

Lemma A.9 (Marginal mass transfers to dominant-code frequency). *Define $\hat{\pi}_k := \frac{1}{B} \sum_{i=1}^B \mathbb{I}[k(i) = k]$. Under Assumption A.5, we have $\|\bar{\mathbf{p}} - \hat{\pi}\|_1 \leq \varepsilon$, hence for all k ,*

$$\hat{\pi}_k \geq \bar{p}_k - \varepsilon. \quad (25)$$

Proof.

$$\|\bar{\mathbf{p}} - \hat{\pi}\|_1 = \left\| \frac{1}{B} \sum_{i=1}^B (\mathbf{p}_i - \mathbf{e}_{k(i)}) \right\|_1 \leq \frac{1}{B} \sum_{i=1}^B \|\mathbf{p}_i - \mathbf{e}_{k(i)}\|_1 \leq \varepsilon.$$

The coordinate-wise bound follows from $|\bar{p}_k - \hat{\pi}_k| \leq \|\bar{\mathbf{p}} - \hat{\pi}\|_1$. $\square$

We will use the following prototype-proximity bound implied by sharp assignments: for any i ,

$$\|\mathbf{z}_i - \mathbf{c}_{k(i)}\|_2 \leq \left(\max_k \|\mathbf{c}_k\|_2 \right) \|\mathbf{p}_i - \mathbf{e}_{k(i)}\|_1 \leq M\varepsilon, \quad (26)$$

where $M := \max_k \|\mathbf{c}_k\|_2$.

Theorem A.10 (Explicit variance lower bound / non-collapse). *Under Assumptions A.4–A.7, let β be defined as in Assumption A.7 (i.e., $\beta = \frac{1 - \rho^*}{K - 1}$), and define*

$$\alpha := \max\{\beta - \varepsilon, 0\}.$$

Let (m, r) be the indices specified in Assumption A.7. If $\alpha > 0$ and $2M\varepsilon < \Delta_c$, then the batch covariance of $\{\mathbf{z}_i\}_{i=1}^B$ obeys

$$\text{Tr}(\text{Cov}(\{\mathbf{z}_i\})) \geq \alpha^2 (\Delta_c - 2M\varepsilon)^2 > 0. \quad (27)$$

Proof. Step 1 (two codes have non-trivial dominant frequency). Let $A = \{i : k(i) = m\}$ and $R = \{i : k(i) = r\}$. By Lemma A.8, $\bar{p}_r \geq \beta$. Since $m = \arg \max_k \bar{p}_k$, we also have $\bar{p}_m \geq \bar{p}_r \geq \beta$. Applying Lemma A.9 yields:

$$\Pr(i \in A) = \hat{\pi}_m \geq \bar{p}_m - \varepsilon \geq \beta - \varepsilon = \alpha,$$

$$\Pr(i \in R) = \hat{\pi}_r \geq \bar{p}_r - \varepsilon \geq \beta - \varepsilon = \alpha.$$

Step 2 (dominant-code membership implies proximity to the corresponding prototype). By (26) and Assumption A.5, we have tight concentration around prototypes:

$$i \in A \implies \|\mathbf{z}_i - \mathbf{c}_m\|_2 \leq M\varepsilon, \quad j \in R \implies \|\mathbf{z}_j - \mathbf{c}_r\|_2 \leq M\varepsilon.$$

Step 3 (two separated prototype-centered clusters imply positive covariance trace). For any $i \in A$ and $j \in R$, the triangle inequality gives:

$$\begin{aligned} \|\mathbf{z}_i - \mathbf{z}_j\|_2 &\geq \|\mathbf{c}_m - \mathbf{c}_r\|_2 - \|\mathbf{z}_i - \mathbf{c}_m\|_2 - \|\mathbf{z}_j - \mathbf{c}_r\|_2 \\ &\geq \Delta_c - 2M\varepsilon. \end{aligned}$$

Now view $\mathbf{Z}$ as a random variable obtained by uniformly sampling an index from the batch, and let $\mathbf{Z}'$ be an independent copy. The pairwise-variance identity yields $\text{Tr}(\text{Cov}(\mathbf{Z})) = \frac{1}{2} \mathbb{E}[\|\mathbf{Z} - \mathbf{Z}'\|_2^2]$. Restricting the expectation to the event $\{i \in A, j \in R\}$ yields the bound:

$$\begin{aligned} \text{Tr}(\text{Cov}(\mathbf{Z})) &\geq \frac{1}{2} \cdot 2 \Pr(i \in A) \Pr(j \in R) (\Delta_c - 2M\varepsilon)^2 \\ &\geq \alpha^2 (\Delta_c - 2M\varepsilon)^2. \end{aligned}$$

If $\alpha > 0$ and $2M\varepsilon < \Delta_c$, the right-hand side is strictly positive, ruling out collapse. $\square$

Remark. Theorem A.10 provides a certificate against collapse. The bound becomes vacuous only if (i) $\alpha = 0$ (low batch entropy), or (ii) $\Delta_c \leq 2M\varepsilon$ (low sharpness or prototype separation). However, these conditions are explicitly encouraged by the batch-entropy, sample-entropy, and prototype-alignment objectives introduced in Section 3.4.

B. Reproducibility Details

B.1. Implementation Details

Data Preprocessing. For each dataset, we identify and remove zero-variance channels using only the official training split, and apply the same channel mask to the corresponding test split. The remaining variables are normalized independently within each window using RevIN. We construct context–target pairs from consecutive non-overlapping windows of length 100, where the current window $\mathbf{X}_t$ serves as input and the subsequent window $\mathbf{X}_{t+1}$ as the prediction target. Each window is partitioned into $P=5$ non-overlapping patches of length $L=20$. For SMD, each server machine is processed independently, and the resulting windows are aggregated for training and evaluation.

All methods are implemented in PyTorch and trained on a single NVIDIA GeForce RTX 4090 GPU. We use the Adam optimizer throughout and, unless otherwise specified, keep the reported hyperparameters fixed across datasets for controlled comparison. We set the latent dimension to $D=256$ and the codebook size to $K=128$, and maintain EMA target networks for both the encoder and codebook.

After each optimization step on the online parameters θ , the corresponding target parameters ξ are updated as

$$\xi \leftarrow \rho\xi + (1 - \rho)\theta,$$

with decay $\rho=0.996$. Gradients are not propagated through the EMA target branch.

Model Architectures.

- **Encoder:** Patch size $L=20$. The projection head is a 2-layer MLP (hidden $64 \rightarrow 32$) mapping to latent dimension $D=256$. The backbone is a 6-layer Transformer (8 heads, dropout 0.1).
- **Codebook:** Codebook size $K=128$, dimension $D=256$. Soft quantization temperature $\tau=0.1$.
- **Predictors:** Both fine and coarse predictors are 2-layer Transformers (4 heads, hidden dimension 128).

Optimization & Hyperparameters.

- **Training:** Adam optimizer ($\text{lr } 5 \times 10^{-4}$, weight decay 10^{-5}), batch size 128. Target networks are updated via EMA (decay 0.996). Gradient clipping norm is 0.5. We train for a maximum of 100 epochs, initiating model selection after 50 epochs with an early stopping patience of 10.
- **Loss Weights:** Predicted objective weights $\lambda_f=1.0$, $\lambda_c=0.5$, and $\gamma=0.1$. Codebook alignment weights $\lambda_{\text{emb}}=1.0$ and $\lambda_{\text{com}}=0.25$. Entropy regularization weights $\lambda_{\text{ent}}^{\text{sample}}=0.005$ and $\lambda_{\text{ent}}^{\text{batch}}=0.01$. Reconstruction weight λ_r is annealed linearly from 0.5 to 0.1 during training.

B.2. SC-JEPA Pre-training Algorithm

We provide the detailed formulation of the total training objective for reproducibility. Following Section 3.4, the total loss is

$$\mathcal{L} = \underbrace{\lambda_f (\mathcal{L}_{\text{KL}}^{\text{fine}} + \gamma \mathcal{L}_{\text{MSE}}^{\text{fine}})}_{\text{Predictive Objective}} + \underbrace{\lambda_{\text{emb}} \mathcal{L}_{\text{emb}} + \lambda_{\text{com}} \mathcal{L}_{\text{com}} + \lambda_{\text{ent}}^{\text{sample}} \mathcal{L}_{\text{ent}}^{\text{sample}} - \lambda_{\text{ent}}^{\text{batch}} \mathcal{L}_{\text{ent}}^{\text{batch}}}_{\text{Codebook Objective}} + \underbrace{\lambda_r \mathcal{L}_{\text{rec}}}_{\text{Reconstruction}}.$$

Algorithm 1 summarizes the pre-training procedure.

Algorithm 1 SC-JEPA Pre-training Procedure

Require: Dataset $\mathcal{D}$, online parameters θ , target parameters ξ , EMA decay ρ

- 1: **while** not converged **do**
 - 2: Sample $(\mathbf{X}_t, \mathbf{X}_{t+1}) \sim \mathcal{D}$ ▷ context–future pair
 - 3: $\mathbf{X}_{t+1}^{\text{coarse}} \leftarrow \text{Downsample}(\mathbf{X}_{t+1})$ ▷ coarse future view
 - 4: $\mathbf{h}_t \leftarrow E_{\theta}(\mathbf{X}_t)$
 - 5: $\mathbf{p}_t, \mathbf{z}_t \leftarrow Q_{\theta}(\mathbf{h}_t)$ ▷ online soft codes
 - 6: $\tilde{\mathbf{X}}_t \leftarrow D_{\theta}(\mathbf{z}_t)$ ▷ reconstruction branch
 - 7: $\hat{\mathbf{p}}_{t+1}^{\text{fine}}, \hat{\mathbf{z}}_{t+1} \leftarrow P_{\theta}^{\text{fine}}(\mathbf{p}_t)$ ▷ fine prediction
 - 8: $\hat{\mathbf{p}}_{t+1}^{\text{coarse}} \leftarrow P_{\theta}^{\text{coarse}}(\mathbf{p}_t)$ ▷ coarse prediction
 - 9: $\mathbf{h}_{t+1}^{\text{fine}} \leftarrow E_{\xi}(\mathbf{X}_{t+1})$
 - 10: $\mathbf{p}_{t+1}^{\text{fine}}, \mathbf{z}_{t+1} \leftarrow Q_{\xi}(\mathbf{h}_{t+1}^{\text{fine}})$ ▷ stop-gradient fine target
 - 11: $\mathbf{h}_{t+1}^{\text{coarse}} \leftarrow E_{\xi}(\mathbf{X}_{t+1}^{\text{coarse}})$
 - 12: $\mathbf{p}_{t+1}^{\text{coarse}} \leftarrow Q_{\xi}(\mathbf{h}_{t+1}^{\text{coarse}})$ ▷ stop-gradient coarse target
 - 13: Compute $\mathcal{L}_{\text{pred}}, \mathcal{L}_{\text{code}}$, and $\mathcal{L}_{\text{rec}}$
 - 14: $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{code}} + \lambda_r \mathcal{L}_{\text{rec}}$
 - 15: Update θ using $\mathcal{L}_{\text{total}}$ ▷ gradient update
 - 16: $\xi \leftarrow \rho \xi + (1 - \rho) \theta$ ▷ EMA target update
 - 17: **end while**
-

B.3. Downstream Anomaly Prediction Protocol

Problem Formulation. We formulate the downstream task as Anomaly Prediction, distinguishing it from standard point-wise anomaly detection. The objective is to utilize a historical context window $\mathbf{X}_t$ to predict the anomaly status of the subsequent non-overlapping future window $\mathbf{X}_{t+1}$. This setup evaluates the model’s ability to identify early warning signs (precursors) rather than merely reacting to ongoing failures. Each dataset provides anomaly annotations as a binary indicator array aligned with the time axis, denoted by $\mathbf{a} \in \{0, 1\}^{T_{\text{full}}}$. Let $\mathcal{I}_{t+1}$ be the set of time indices covered by the future window $\mathbf{X}_{t+1}$. We define the window-level label as

$$y_{t+1} = \mathbb{I} \left(\sum_{\tau \in \mathcal{I}_{t+1}} \mathbf{a}_{\tau} > 0 \right) \in \{0, 1\}. \quad (28)$$

Let $\hat{s}_{t+1} \in [0, 1]$ denote the predicted anomaly probability and let

$$\hat{y}_{t+1} = \mathbb{I}(\hat{s}_{t+1} > \delta^*) \quad (29)$$

be the binary prediction using threshold δ^* selected on the validation set to maximize the window-level F1 score.

Notation. We denote by $\mathbf{X}_t$ the context window and by $\mathbf{X}_{t+1}$ the subsequent non-overlapping future window. The supervision signal is the *future-window* label $y_{t+1} \in \{0, 1\}$, which indicates whether $\mathbf{X}_{t+1}$ is anomalous under the window-level definition above. Thus, each downstream training example is a pair $(\mathbf{X}_t, y_{t+1})$, and the goal is anomaly prediction.

We report window-level Precision, Recall, and F1-score:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (30)$$

We additionally report the Area Under the Receiver Operating Characteristic curve (AUC) computed from the raw probabilities $\hat{s}_{t+1}$.

Adaptation Strategy. We utilize the pre-trained SC-JEPA as a frozen feature extractor. Specifically, we discard the projection heads and freeze the parameters of the encoder and codebook. For a given input $\mathbf{X}_t$, the encoder yields soft code probabilities $\mathbf{p}_t \in \mathbb{R}^{V \times P \times K}$, where V denotes variables, P patches, and K the codebook size. To capture the most salient anomalies across the multivariate dimensions while preserving temporal structure, we apply *Variable-wise Max-Pooling*:

$$\mathbf{h}_t = \max_{v \in \{1 \dots V\}} \mathbf{p}_t^{(v)} \in \mathbb{R}^{P \times K}. \quad (31)$$

The aggregated representation $\mathbf{h}_t$ is flattened and passed to an MLP classifier C_ψ , trained to estimate $\hat{s}_{t+1} = P(y_{t+1} = 1 \mid \mathbf{h}_t)$.

Algorithm 2 Downstream Anomaly Prediction Protocol

Require: Frozen encoder E_θ , frozen codebook Q , and splits $\mathcal{S}_{\text{tr}}$, $\mathcal{S}_{\text{val}}$, $\mathcal{S}_{\text{te}}$

- 1: Initialize MLP classifier C_ψ
 - 2: **for** each batch $(\mathbf{X}_t, y_{t+1}) \in \mathcal{S}_{\text{tr}}$ **do**
 - 3: $\mathbf{p}_t \leftarrow Q(E_\theta(\mathbf{X}_t))$ $\triangleright \mathbf{p}_t \in \mathbb{R}^{V \times P \times K}$
 - 4: $\mathbf{h}_t \leftarrow \max_v \mathbf{p}_t^{(v)}$ $\triangleright \mathbf{h}_t \in \mathbb{R}^{P \times K}$
 - 5: $\hat{s}_{t+1} \leftarrow C_\psi(\mathbf{h}_t)$
 - 6: Update ψ using BCE($\hat{s}_{t+1}, y_{t+1}$)
 - 7: **end for**
 - 8: Select δ^* on $\mathcal{S}_{\text{val}}$ to maximize window-level F1
 - 9: Evaluate Precision, Recall, and F1 on $\mathcal{S}_{\text{te}}$ using δ^* , and AUC using $\hat{s}_{t+1}$
-

C. Experimental Results

C.1. Main Benchmark

Table 6 presents the comprehensive performance evaluation of our proposed method against baseline models across standard anomaly detection benchmarks: MSL, SMAP, SWaT, SMD and PSM. In addition to the F1-score and AUC reported in the main text, we provide detailed comparisons on Precision and Recall to offer a holistic view of the detection capabilities. All metrics are reported as the mean $\pm$ standard deviation over five independent runs. The results demonstrate that our method consistently achieves superior accuracy and robustness compared to both classical unsupervised baselines (e.g., K-Means, DeepSVDD) and recent state-of-the-art multivariate time-series models (e.g., PatchTST, TS2Vec).

Table 6. Full results on five benchmark datasets. All metrics are reported in percentage (%) as mean $\pm$ standard deviation over 5 runs with different random seeds. Best baselines are underlined.

Models	MSL				SMAP				SWaT				PSM				SMD			
	F1	AUC	Prec	Rec	F1	AUC	Prec	Rec	F1	AUC	Prec	Rec	F1	AUC	Prec	Rec	F1	AUC	Prec	Rec
K-Means	20.63 $\pm$ 2.30	52.17 $\pm$ 5.65	19.27 $\pm$ 7.16	31.34 $\pm$ 15.40	8.74 $\pm$ 3.39	39.82 $\pm$ 2.57	10.52 $\pm$ 8.17	10.16 $\pm$ 4.92	14.45 $\pm$ 0.38	62.20 $\pm$ 0.16	7.99 $\pm$ 0.32	77.05 $\pm$ 8.16	42.75 $\pm$ 1.73	51.37 $\pm$ 3.80	30.40 $\pm$ 2.34	78.89 $\pm$ 21.18	14.79 $\pm$ 5.41	57.49 $\pm$ 0.22	35.83 $\pm$ 34.03	20.64 $\pm$ 11.95
DeepSVD	23.46 $\pm$ 0.98	53.67 $\pm$ 4.80	14.78 $\pm$ 2.14	73.87 $\pm$ 26.12	22.05 $\pm$ 2.97	44.07 $\pm$ 2.61	12.90 $\pm$ 1.19	<u>81.99 $\pm$ 24.23</u>	15.28 $\pm$ 1.75	58.22 $\pm$ 2.34	12.65 $\pm$ 1.90	20.00 $\pm$ 3.34	44.45 $\pm$ 1.56	49.95 $\pm$ 3.10	30.31 $\pm$ 3.33	89.38 $\pm$ 16.00	9.41 $\pm$ 2.42	47.31 $\pm$ 4.44	6.25 $\pm$ 1.61	19.15 $\pm$ 4.85
LSTM-VAE	22.42 $\pm$ 7.02	55.22 $\pm$ 10.27	20.84 $\pm$ 10.10	32.43 $\pm$ 15.96	22.77 $\pm$ 1.73	52.97 $\pm$ 2.60	14.35 $\pm$ 1.21	55.39 $\pm$ 4.47	54.60 $\pm$ 1.17	79.91 $\pm$ 1.00	64.92 $\pm$ 14.30	50.07 $\pm$ 8.30	56.73 $\pm$ 1.14	71.95 $\pm$ 0.50	49.32 $\pm$ 1.59	66.86 $\pm$ 2.10	10.04 $\pm$ 1.10	56.18 $\pm$ 0.86	5.78 $\pm$ 0.58	38.29 $\pm$ 6.92
TTransformer	27.25 $\pm$ 2.03	64.61 $\pm$ 1.19	16.71 $\pm$ 1.25	75.40 $\pm$ 11.80	33.00 $\pm$ 2.05	60.91 $\pm$ 1.58	22.67 $\pm$ 2.83	64.32 $\pm$ 10.77	70.49 $\pm$ 1.00	82.10 $\pm$ 0.36	98.17 $\pm$ 1.82	55.09 $\pm$ 1.18	54.12 $\pm$ 0.92	63.99 $\pm$ 0.75	37.37 $\pm$ 1.17	98.29 $\pm$ 2.10	14.20 $\pm$ 0.75	57.94 $\pm$ 0.85	8.11 $\pm$ 0.76	<u>62.24 $\pm$ 13.63</u>
TimesNet	28.44 $\pm$ 5.79	59.70 $\pm$ 5.15	30.31 $\pm$ 7.40	65.59 $\pm$ 27.00	27.78 $\pm$ 1.48	57.01 $\pm$ 1.81	17.98 $\pm$ 0.60	62.81 $\pm$ 10.10	67.20 $\pm$ 3.94	83.24 $\pm$ 1.64	90.87 $\pm$ 15.87	54.42 $\pm$ 6.09	52.64 $\pm$ 0.02	56.12 $\pm$ 2.66	35.76 $\pm$ 0.09	99.21 $\pm$ 0.57	12.51 $\pm$ 0.49	56.33 $\pm$ 0.87	7.64 $\pm$ 0.42	35.96 $\pm$ 6.64
Qwen2.5-3B	21.24 $\pm$ 0.52	55.66 $\pm$ 0.84	17.82 $\pm$ 1.28	26.67 $\pm$ 2.22	0.34 $\pm$ 0.69	63.61 $\pm$ 1.05	1.67 $\pm$ 3.33	0.19 $\pm$ 0.38	17.29 $\pm$ 1.85	56.75 $\pm$ 7.35	41.82 $\pm$ 21.26	14.43 $\pm$ 6.83	47.44 $\pm$ 9.10	65.65 $\pm$ 5.93	50.44 $\pm$ 6.09	46.57 $\pm$ 14.36	13.00 $\pm$ 1.78	60.66 $\pm$ 4.59	8.29 $\pm$ 1.41	31.06 $\pm$ 3.65
PAD	21.67 $\pm$ 0.03	55.97 $\pm$ 6.96	12.19 $\pm$ 0.06	92.28 $\pm$ 4.44	26.92 $\pm$ 1.78	59.79 $\pm$ 0.30	18.15 $\pm$ 1.21	55.00 $\pm$ 11.80	69.75 $\pm$ 4.83	83.71 $\pm$ 0.61	81.54 $\pm$ 15.17	62.79 $\pm$ 4.32	57.82 $\pm$ 5.96	73.84 $\pm$ 1.85	57.54 $\pm$ 3.18	58.57 $\pm$ 9.17	12.52 $\pm$ 0.34	58.53 $\pm$ 2.35	6.71 $\pm$ 0.20	93.40 $\pm$ 2.46
A2P	23.10 $\pm$ 0.85	59.43 $\pm$ 1.28	13.92 $\pm$ 1.18	74.12 $\pm$ 17.29	21.68 $\pm$ 4.10	60.37 $\pm$ 1.42	15.88 $\pm$ 1.26	38.85 $\pm$ 18.15	70.18 $\pm$ 0.26	81.92 $\pm$ 0.10	92.10 $\pm$ 0.74	54.33 $\pm$ 0.33	50.71 $\pm$ 0.96	57.01 $\pm$ 0.38	37.16 $\pm$ 0.97	80.00 $\pm$ 3.48	15.21 $\pm$ 0.73	58.87 $\pm$ 1.55	12.23 $\pm$ 1.48	22.13 $\pm$ 6.33
FCM	24.57 $\pm$ 0.73	61.09 $\pm$ 0.76	14.25 $\pm$ 0.54	90.00 $\pm$ 7.37	30.60 $\pm$ 3.08	61.85 $\pm$ 0.86	25.25 $\pm$ 2.76	41.41 $\pm$ 9.44	69.09 $\pm$ 1.08	81.66 $\pm$ 0.37	94.25 $\pm$ 7.92	54.83 $\pm$ 1.86	52.34 $\pm$ 0.89	57.25 $\pm$ 1.18	37.00 $\pm$ 1.39	89.85 $\pm$ 4.37	10.25 $\pm$ 1.46	60.44 $\pm$ 0.65	15.59 $\pm$ 2.43	28.94 $\pm$ 10.43
MICN	20.00 $\pm$ 0.96	60.18 $\pm$ 2.26	11.56 $\pm$ 0.57	75.29 $\pm$ 10.12	20.81 $\pm$ 2.38	51.16 $\pm$ 1.72	13.75 $\pm$ 3.01	55.58 $\pm$ 22.01	48.31 $\pm$ 3.19	79.38 $\pm$ 0.30	77.99 $\pm$ 7.96	35.33 $\pm$ 3.71	48.61 $\pm$ 1.38	54.97 $\pm$ 1.12	34.64 $\pm$ 1.33	82.09 $\pm$ 6.61	12.05 $\pm$ 0.70	52.21 $\pm$ 0.37	7.72 $\pm$ 0.53	28.72 $\pm$ 5.83
Pathformer	21.69 $\pm$ 2.29	61.86 $\pm$ 0.66	13.14 $\pm$ 1.43	68.24 $\pm$ 18.45	23.58 $\pm$ 1.27	58.67 $\pm$ 0.38	14.10 $\pm$ 0.68	72.12 $\pm$ 7.37	14.90 $\pm$ 1.72	59.87 $\pm$ 1.24	9.20 $\pm$ 1.04	40.98 $\pm$ 9.16	51.77 $\pm$ 0.84	54.41 $\pm$ 3.09	35.04 $\pm$ 0.76	99.10 $\pm$ 1.19	14.32 $\pm$ 0.50	56.49 $\pm$ 1.71	9.41 $\pm$ 1.24	36.60 $\pm$ 15.82
TS-JEPA	25.49 $\pm$ 4.22	60.33 $\pm$ 4.65	17.10 $\pm$ 3.91	56.84 $\pm$ 18.65	26.57 $\pm$ 1.67	57.38 $\pm$ 0.72	18.23 $\pm$ 0.74	49.73 $\pm$ 8.05	71.95 $\pm$ 0.56	80.93 $\pm$ 0.69	84.32 $\pm$ 1.67	62.76 $\pm$ 0.33	53.32 $\pm$ 2.25	66.10 $\pm$ 3.11	36.95 $\pm$ 2.50	96.29 $\pm$ 2.65	7.49 $\pm$ 2.65	49.53 $\pm$ 8.44	19.11 $\pm$ 27.96	20.43 $\pm$ 14.37
PatchTST	26.98 $\pm$ 0.51	60.43 $\pm$ 1.13	75.00 $\pm$ 0.00	16.45 $\pm$ 0.38	30.06 $\pm$ 1.20	<u>61.62 $\pm$ 0.66</u>	21.76 $\pm$ 0.93	49.06 $\pm$ 2.10	70.64 $\pm$ 3.57	81.93 $\pm$ 1.27	94.03 $\pm$ 4.25	57.58 $\pm$ 3.94	<u>58.17 $\pm$ 0.73</u>	<u>75.76 $\pm$ 0.11</u>	<u>64.68 $\pm$ 0.59</u>	52.86 $\pm$ 0.90	11.79 $\pm$ 0.43	47.76 $\pm$ 0.95	6.58 $\pm$ 0.22	57.02 $\pm$ 3.40
TS2Vec	23.48 $\pm$ 2.14	<u>64.86 $\pm$ 0.54</u>	31.45 $\pm$ 2.46	18.82 $\pm$ 2.35	32.81 $\pm$ 0.32	61.48 $\pm$ 0.46	<u>23.81 $\pm$ 0.30</u>	52.81 $\pm$ 1.95	67.00 $\pm$ 0.83	<u>83.76 $\pm$ 0.23</u>	80.95 $\pm$ 2.10	57.17 $\pm$ 0.41	48.43 $\pm$ 1.01	72.13 $\pm$ 0.43	32.39 $\pm$ 0.58	95.96 $\pm$ 2.90	14.78 $\pm$ 1.46	56.28 $\pm$ 0.31	8.83 $\pm$ 1.28	50.43 $\pm$ 11.51
SC-JEPA	<u>33.58 $\pm$ 4.34</u>	<u>66.08 $\pm$ 3.25</u>	<u>35.87 $\pm$ 10.90</u>	<u>40.80 $\pm$ 15.68</u>	<u>33.64 $\pm$ 1.45</u>	<u>65.41 $\pm$ 2.06</u>	<u>24.24 $\pm$ 1.28</u>	<u>56.02 $\pm$ 8.18</u>	<u>72.89 $\pm$ 0.70</u>	<u>84.95 $\pm$ 0.82</u>	<u>98.00 $\pm$ 1.00</u>	<u>58.05 $\pm$ 1.20</u>	<u>61.61 $\pm$ 4.32</u>	<u>77.85 $\pm$ 1.28</u>	<u>55.01 $\pm$ 2.47</u>	<u>72.00 $\pm$ 12.96</u>	<u>20.82 $\pm$ 2.61</u>	<u>62.29 $\pm$ 0.90</u>	<u>18.18 $\pm$ 5.24</u>	31.49 $\pm$ 13.48

C.2. Generality

We evaluate generality by comparing in-domain and cross-domain pre-training while keeping the downstream evaluation exactly the same.

For a target dataset, in-domain pre-training uses its official training set. Cross-domain pre-training uses the union of the other three datasets, with the target dataset completely excluded during pre-training. Since the source datasets may have different channel dimensions in this cross-domain setting, we treat each channel as an independent univariate series during pre-training, which avoids any explicit channel matching across datasets. After pre-training, we follow the same downstream protocol and evaluate on the same target test split in both settings, so the only difference is the source of pre-training data.

Table 7. Comparison of in-domain versus cross-domain generalization.

Models	Settings	MSL		SMAP		SWaT		PSM	
		F1	AUC	F1	AUC	F1	AUC	F1	AUC
PatchTST	In-Domain	26.98 $\pm$ 0.51	60.43 $\pm$ 1.13	30.06 $\pm$ 1.20	61.62 $\pm$ 0.66	70.64 $\pm$ 3.57	81.93 $\pm$ 1.27	58.17 $\pm$ 0.73	75.76 $\pm$ 0.11
	Cross-Domain	27.08 $\pm$ 0.44	60.80 $\pm$ 1.17	26.21 $\pm$ 0.24	58.77 $\pm$ 1.63	54.34 $\pm$ 5.96	68.76 $\pm$ 0.87	50.07 $\pm$ 0.19	53.72 $\pm$ 0.93
	Abs. Diff.	+0.10	+0.37	-3.85	-2.85	-16.30	-13.17	-8.10	-22.04
	Rel. Diff.	+0.37%	+0.61%	-12.81%	-4.63%	-23.07%	-16.07%	-13.92%	-29.09%
TS2Vec	In-Domain	23.48 $\pm$ 2.14	64.86 $\pm$ 0.54	32.81 $\pm$ 0.32	61.48 $\pm$ 0.46	67.00 $\pm$ 0.83	83.76 $\pm$ 0.23	48.43 $\pm$ 1.01	72.13 $\pm$ 0.43
	Cross-Domain	22.87 $\pm$ 2.67	62.00 $\pm$ 0.74	26.99 $\pm$ 0.75	57.54 $\pm$ 0.35	65.08 $\pm$ 0.53	82.91 $\pm$ 0.02	48.15 $\pm$ 7.73	56.22 $\pm$ 3.49
	Abs. Diff.	-0.61	-2.86	-5.82	-3.94	-1.92	-0.85	-0.28	-15.91
	Rel. Diff.	-2.60%	-4.41%	-17.74%	-6.41%	-2.87%	-1.01%	-0.58%	-22.06%
SC-JEPA	In-Domain	33.58 $\pm$ 4.34	66.08 $\pm$ 3.25	33.64 $\pm$ 1.45	65.41 $\pm$ 2.06	72.89 $\pm$ 0.70	84.95 $\pm$ 0.82	61.61 $\pm$ 4.32	77.85 $\pm$ 1.28
	Cross-Domain	33.48 $\pm$ 7.26	67.13 $\pm$ 6.15	33.81 $\pm$ 5.13	68.88 $\pm$ 1.45	71.10 $\pm$ 0.76	82.96 $\pm$ 0.40	56.87 $\pm$ 4.31	66.80 $\pm$ 6.02
	Abs. Diff.	-0.10	+1.05	+0.17	+3.47	-1.79	-1.99	-4.74	-11.05
	Rel. Diff.	-0.30%	+1.59%	+0.51%	+5.30%	-2.46%	-2.34%	-7.69%	-14.19%

C.3. Ablation Studies

Table 8 reports the complete ablation results. All variants follow the same training and evaluation protocol as the full model, and we modify only the targeted component while keeping the rest of the architecture and training pipeline unchanged. For the codebook operation studies, the full model uses a discrete codebook bottleneck that represents each latent via soft assignment to a finite set of prototypes, with a slowly updated EMA target branch providing a stable reference for prototype states and assignment targets. To separate the effect of the bottleneck structure from the effect of auxiliary constraints, we evaluate two complementary interventions. In the loss-only removal setting, we keep the soft codebook bottleneck in both the online and EMA target branches, and we set the auxiliary codebook regularizer weights to zero. In the module removal setting, we remove the codebook bottleneck from both branches. The model then operates purely in continuous latent space, and the predictive objective reduces to an MSE loss because the code-assignment KL term is no longer defined without discrete code distributions.

Note. When the standard deviation across random seeds is (near) zero while the mean score is low, this typically indicates model collapse, i.e., the model makes nearly identical (often single-class) predictions across runs and loses discriminative signal.

Table 8. Ablation results on five benchmark datasets.

Datasets	MSL		SMAP		SWaT		PSM		SMD	
Metrics	F1	AUC	F1	AUC	F1	AUC	F1	AUC	F1	AUC
w/o KL Regularization	29.01 ± 6.64	53.90 ± 5.25	28.71 ± 1.71	60.19 ± 0.95	67.33 ± 3.97	79.95 ± 0.97	54.93 ± 2.54	70.83 ± 1.25	12.63 ± 3.75	53.61 ± 8.33
w/o Reconstruction Decoder	25.96 ± 1.82	52.68 ± 1.68	25.15 ± 0.31	52.80 ± 1.88	14.77 ± 1.42	53.30 ± 2.45	53.03 ± 0.00	69.84 ± 7.74	10.18 ± 1.65	50.44 ± 1.44
w/o Predictive Objective	28.88 ± 6.41	53.17 ± 1.03	27.05 ± 2.31	59.81 ± 1.76	59.47 ± 1.55	78.60 ± 0.59	58.89 ± 5.50	70.17 ± 4.73	10.04 ± 3.42	50.24 ± 5.07
w/o Codebook Loss	31.62 ± 1.71	58.93 ± 1.81	31.53 ± 7.88	62.06 ± 0.60	72.64 ± 1.24	82.63 ± 0.63	60.83 ± 4.09	75.85 ± 0.65	12.82 ± 2.01	57.36 ± 4.25
w/o Codebook Module	21.82 ± 0.00	43.02 ± 9.30	21.69 ± 0.00	51.00 ± 1.23	11.51 ± 0.00	50.00 ± 0.00	53.03 ± 0.00	46.61 ± 6.79	12.45 ± 0.00	50.00 ± 0.00
w/o Downsampling	28.77 ± 6.95	63.16 ± 3.83	30.39 ± 2.47	61.34 ± 0.15	71.62 ± 0.28	83.60 ± 0.40	60.79 ± 3.31	71.72 ± 4.68	15.50 ± 1.54	60.93 ± 0.10
Full Model	33.58 ± 4.34	66.08 ± 3.25	33.64 ± 1.45	65.41 ± 2.06	72.89 ± 0.70	84.95 ± 0.82	61.61 ± 4.32	77.85 ± 1.28	20.82 ± 2.61	62.29 ± 0.90

D. Visualizations and Qualitative Analysis

D.1. Framework

Figure 5 illustrates the end-to-end data flow and objectives of our framework, which consists of five coordinated components:

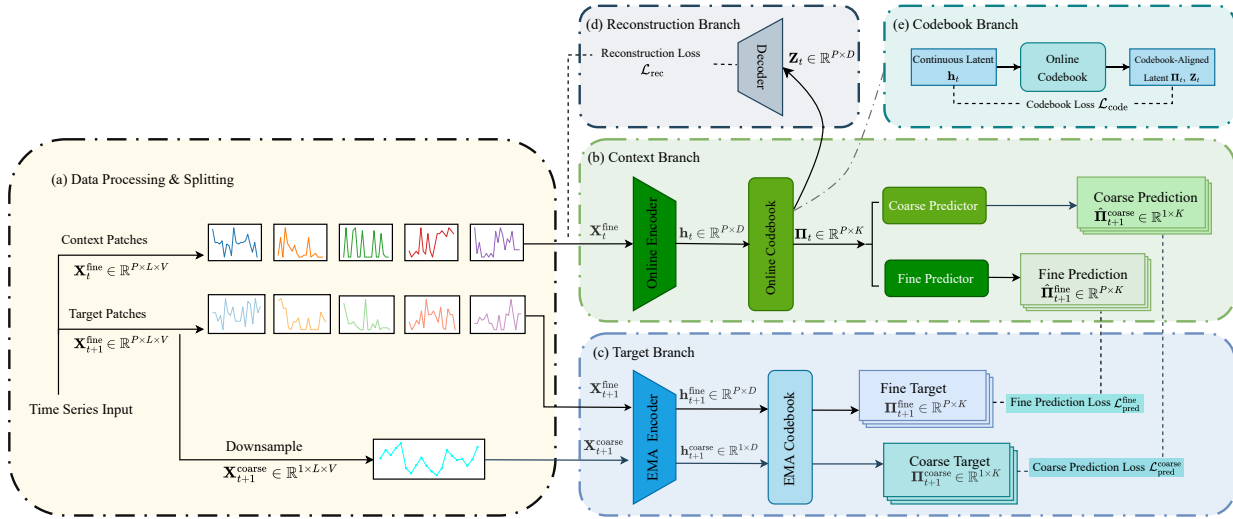


Figure 5. Overview of the Proposed Framework.

- Data Processing & Splitting:** We construct consecutive context–target pairs from non-overlapping windows. The context window is tokenized into fine-grained patches $\mathbf{X}_t^{\text{fine}} \in \mathbb{R}^{P \times L \times V}$. Simultaneously, the future window is processed into two views: fine-grained patches $\mathbf{X}_{t+1}^{\text{fine}}$ and a downsampled coarse view $\mathbf{X}_{t+1}^{\text{coarse}}$, obtained by averaging every P consecutive time points to capture low-frequency trends.
- Context Branch:** The online encoder processes the context patches $\mathbf{X}_t^{\text{fine}}$ to produce continuous latents $\mathbf{h}_t$. These are passed through the Online Codebook to generate the discrete code distributions $\Pi_t \in \mathbb{R}^{P \times K}$. This sequence Π_t is then shared by two predictors: the *Fine Predictor* outputs patch-level predictions $\hat{\Pi}_{t+1}^{\text{fine}}$, while the *Coarse Predictor* aggregates the history to produce a global prediction $\hat{\Pi}_{t+1}^{\text{coarse}}$.
- Target Branch:** To provide stable supervision, the momentum-updated EMA encoder processes both future views ($\mathbf{X}_{t+1}^{\text{fine}}$ and $\mathbf{X}_{t+1}^{\text{coarse}}$). The resulting features are mapped by the EMA Codebook into target distributions Π_{t+1}^{fine} and $\Pi_{t+1}^{\text{coarse}}$. The model minimizes the prediction losses $\mathcal{L}_{\text{pred}}^{\text{fine}}$ and $\mathcal{L}_{\text{pred}}^{\text{coarse}}$ by matching the online predictions to these targets.
- Reconstruction Branch:** To prevent representation collapse and anchor the semantics, a decoder reconstructs the original signal from the codebook-aligned embeddings $\mathbf{Z}_t$, optimizing the reconstruction loss $\mathcal{L}_{\text{rec}}$.
- Codebook Branch:** This module aligns the continuous latents $\mathbf{h}_t$ with the discrete code assignments Π_t and embeddings $\mathbf{Z}_t$. It computes the codebook loss $\mathcal{L}_{\text{code}}$, which regularizes the assignment probabilities to ensure the codebook remains utilized and structured.

D.2. Inference Efficiency

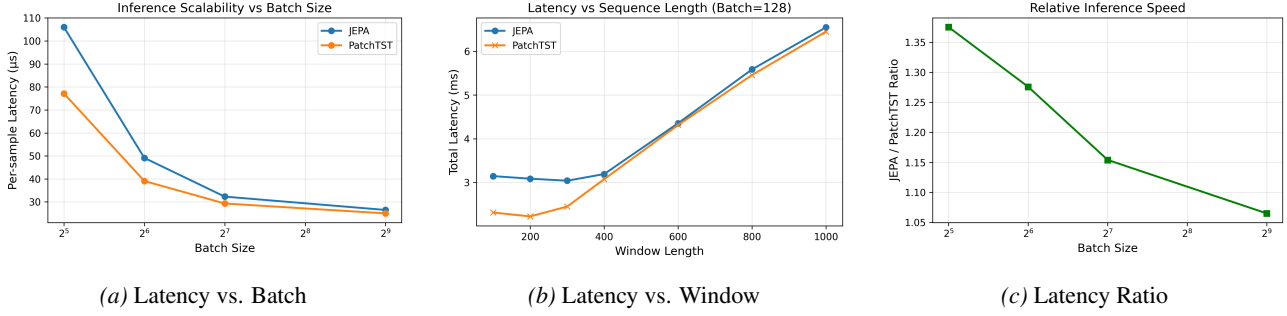


Figure 6. Inference-time scaling comparisons across batch size, window length, and latency ratios.

Figure 6 evaluates the inference efficiency of SC-JEPA compared with PatchTST from three complementary perspectives. First, Figure 6a reports the per-sample latency vs. batch size: SC-JEPA is slower, but the gap shrinks at larger batches. Figure 6b then reports latency vs. window length (batch fixed): both scale similarly, with SC-JEPA slightly slower. Figure 6c confirms this trend: the latency ratio decreases as batch size increases.

Overall, these results highlight a limitation of our approach: inference efficiency is not a primary contribution of SC-JEPA, as its richer architecture incurs higher computational cost than a single-backbone baseline. Our design instead prioritizes early-warning performance and representation stability, with the runtime overhead becoming more manageable in deployment scenarios that support moderate to large batch sizes.